



**Department of Computer Science
Software Engineering**

Phase 2

Software Design Specification Document

[InnovaCode](#)

Name and ID:

Alia Khamis U19102199

Amna Rashed Almazrouei U23103547

Asma Al-Ali U22100601

Hasna Imad U23100997

Hind Almarzooqi U21201139

Namah Sultan U22100470

Reyam Sahib U23105903

Table of Contents

Section 1 – Introduction:	4
1.0 Introduction:	4
1.1 Goals and Objectives:	4
1.2 Statement of Scope:	5
1.3 Software Context:	7
1.4 Major Constraints:	8
Section 2 – Data Design:	9
2.0 Data Design:	9
2.1 Internal Software Data Structure:	9
2.2 Global Data Structure:	10
2.3 Temporary Data Structure:	11
2.4 Database Description	11
2.4.1 Core Database (Tables and Descriptions)	11
2.4.2 AI and Analytics Database	16
Section 3 – Architectural and Component-Level Design:	19
3.0 Architectural and Component-Level Design:	19
3.1 System Structure:	27
3.1.1 Architecture Diagram:	27
3.2 Description for Components	32
3.2 Description for Component User	32
3.2 Description for Component Order	40
3.2 Description for Component Payment	47
3.2 Description for Component Entitlement	53
3.2 Description for Component AI Stock Automation	59
3.2 Description for Component Delivery Status Tracking	66
3.2 Description for Component AI Resource Optimizer	72
3.2 Description for Component Payment Refund Processing	77
3.2 Description for Component AI Recipe Recommendation	83
3.2 Description for Component Sustainability Dashboard	90
3.3 Dynamic Behavior for Components	98
3.3.1 Interaction Diagrams	98
1) User Registration Process	98
2) User Login Process	99
3) Browse Products and Add to Cart	100
4) Complete Order and Payment	101
5) Assign Delivery Partner	102
6) Track Order Delivery	104
7) Shareholder Claims Entitlement	105
8) Investor Funds Farm Project	106
9) AI Generates Demand Forecast	107
10) Submit Support Ticket	110
11) List Surplus Produce	111

12) Cancel Order	112
13) Farmer Uploads Product	113
14) AI Product Recommendations	115
15) Register for Community Event	116
 Section 4 – User Interface Design	117
4.0 User Interface Design	117
4.1 Description of the User Interface	117
4.1.1 Screen Images	117
4.1.1.1 Splash Screen and Onboarding	118
4.1.1.2 Dashboard Pages	118
4.1.1.3 Product & AI Feature Pages	119
4.1.1.4 Investment and Community Pages	119
4.1.2 Objects and Actions	121
4.2 Interface Design Rules	121
4.3 Components Available	122
4.4 UIDS Description	122
 Section 5	123
5.0 Restrictions, Limitations, and Constraints	123
Major Constraints Affecting Design	123
Major Design Limitations	124
 Section 6	125
6.0 Testing Overview	125
6.1 Testing Techniques	125
6.2 Expected Software Response	125
6.3 Types of Testing	126
6.4 Performance Bounds	126
6.5 Identification of Critical Components	128

Section 1 – Introduction:

1.0 Introduction:

This document defines the technical and functional specifications of the **Shared Farm System**, an intelligent web-based platform designed to connect farmers, investors, shareholders, delivery partners, and consumers within a unified digital environment. The document describes the software's purpose, scope, functions, and expected behavior.

The Shared Farm System acts as a transparent agricultural marketplace and management solution. It allows customers to purchase fresh produce directly from verified farms, shareholders to manage their produce entitlements, and investors to participate in sustainable farm projects. The system also supports administrators and community members by integrating logistics, education, and sustainability tracking into a single platform.

The platform emphasizes transparency, automation, and collaboration. By embedding artificial-intelligence-based forecasting and recommendation models, it improves production planning, minimizes waste, and enhances decision-making for all participants. The system operates through a modern, scalable architecture accessible from both desktop and mobile devices.

1.1 Goals and Objectives:

The main goal of the Shared Farm System is to promote community-driven agriculture and enable digital participation in farming, trading, and investment.

The project has the following key objectives:

1. To design an integrated online platform that connects customers, farmers, shareholders, investors, and administrators.
2. To ensure transparent and traceable farm-to-consumer transactions supported by verified data.

3. To introduce AI-powered tools for demand forecasting, inventory automation, and personalized product recommendations.
4. To provide shareholders and investors with real-time dashboards showing productivity, sustainability, and financial indicators.
5. To encourage community education through workshops, events, and resource-sharing features.
6. To establish secure digital payments, refunds, and entitlement management.
7. To support long-term scalability by integrating logistics APIs, payment gateways, and environmental data sources.

1.2 Statement of Scope:

The scope of the Shared Farm System is to build a centralized digital ecosystem that streamlines agricultural production, distribution, and investment. The system facilitates collaboration between multiple roles—customers, shareholders, investors, administrators, delivery partners, and community members—each with specific access privileges.

Customers can browse farm produce, place orders, and receive deliveries directly from verified sources. Shareholders can upload surplus stock, claim entitlements, and monitor farm performance. Investors can fund agricultural projects, view returns, and track sustainability outcomes. Administrators manage users, verify data, and moderate community activities. Delivery partners coordinate logistics and confirm deliveries, while community members engage in workshops and educational content.

The software ensures data accuracy through real-time validation and secure record-keeping. It integrates machine-learning models for forecasting, automated inventory control, and user-centric recommendations to improve efficiency and reduce waste.

The following table ranks the **major processing functions** that define the core scope of the system.

Function	Description	Priority
Creating Accounts and Logging In	Allows all users to register, log in, and access personalized dashboards. Role-based permissions determine available features.	Essential
Product Browsing and Filtering	Lets customers view farm produce, apply filters by type, origin, and price, and access sustainability data.	Essential
Order Placement and Checkout	Handles shopping-cart functions, payment verification, and order confirmation with digital receipts.	Essential
Payment Processing and Refunds	Verifies payments through gateways, authenticates transactions, and manages refund requests.	Essential
Delivery Scheduling and Tracking	Coordinates delivery partners, schedules time slots, and updates delivery status for customers.	Essential
Shareholder Entitlement Management	Enables shareholders to claim produce shares and monitor delivery or transfer of entitlements.	Essential
Surplus Produce Exchange	Allows shareholders to list excess produce for exchange or donation with administrative approval.	Essential
Investment Management	Provides investors access to farm projects, allows secure funding, and tracks financial and environmental returns.	Essential
Sustainability Dashboard	Displays analytics on water usage, carbon savings, and waste reduction metrics for farms.	Desirable
AI Demand Forecasting	Predicts produce demand to plan harvest cycles and minimize waste.	Essential
AI Stock Automation	Monitors real-time stock levels, issues low-stock alerts, and triggers auto-restock processes.	Essential
AI Recommendations and Recipes	Suggests personalized product bundles, recipes, and storage tips based on purchase history.	Essential

AI Resource Optimization	Analyses farm resource usage to recommend efficient allocation of equipment and inputs.	Desirable
Community Workshops and Events	Allows users to register for events, share experiences, and access learning materials.	Essential
Administrative Control	Enables administrators to manage users, approve transactions, and generate reports.	Essential
Customer Support and Chat	Offers chat assistance and automated inquiry responses.	Desirable
Data Validation and Verification	Confirms authenticity of user-submitted data and prevents duplication or false entries.	Essential
Reward and Loyalty System	Awards points for eco-friendly actions redeemable for discounts or event participation.	Future
Third-Party Farm Auditing	Integrates certified auditors for farm inspection and compliance verification.	Future
Fraud Detection and Security	Monitors irregular transaction patterns and flags suspicious activity.	Future

The system's functionality ensures that every role can interact within the same environment while maintaining data integrity, user privacy, and operational transparency.

1.3 Software Context:

The Shared Farm System operates through a layered architecture that separates presentation, business logic, and data management. The front-end interface is developed using responsive web technology compatible with all major browsers and mobile devices. The application layer handles transactions, user authentication, order management, AI computations, and reporting.

All system data—including user information, produce listings, orders, and investment details—is securely stored in cloud databases. Artificial-intelligence modules run parallel to these processes, performing forecasting and recommendation tasks using historical data and real-time updates.

External integration plays a major role in system context. The platform connects with logistics APIs for delivery scheduling, payment gateways for secure transactions, and environmental databases for sustainability metrics. Its design emphasizes scalability, enabling easy addition of new farms, regions, and investors as the network expands.

1.4 Major Constraints:

Although the Shared Farm System is designed to deliver efficiency and transparency, several constraints must be considered:

- **Data Accuracy:** The reliability of AI modules depends on the accuracy of data uploaded by farms and users. Incomplete or outdated records may lead to forecasting errors.
- **Internet Connectivity:** Many farms are located in rural areas with limited network coverage, which may affect system performance.
- **User Adoption:** Farmers and shareholders unfamiliar with digital platforms may require orientation or support to effectively use the system.
- **Integration Challenges:** Incorporating external APIs such as weather data, logistics networks, and payment services can introduce compatibility and maintenance issues.
- **Regulatory Compliance:** Agricultural investment, data privacy, and food-safety regulations differ across jurisdictions and must be carefully followed.
- **Security Risks:** Because the system manages financial transactions, strict encryption and access-control mechanisms are required.
- **Scalability:** As participation increases, server resources and data-handling capacity must scale efficiently.
- **AI Model Limitations:** Predictive accuracy may fluctuate based on historical patterns and environmental changes.

The project will be considered successful if it delivers a functional, secure, and scalable platform that promotes transparency, sustainability, and stakeholder collaboration across the agricultural ecosystem.

Section 2 – Data Design:

2.0 Data Design:

This section defines all internal, global, and temporary data structures that form the foundation of the **Shared Farm System**. Each structure ensures reliable storage, retrieval, and processing of information within the platform.

The data design aims to maintain transparency, scalability, and efficiency across every user interaction — from farm-level operations to AI-driven analytics and community events.

2.1 Internal Software Data Structure:

The internal software structure is composed of interconnected components that collectively define the platform's core logic and business operations. These structures form the operational backbone of Shared Farm.

1. **User Profile** – Represents individuals interacting with the system. Each profile contains identifiers, contact data, credentials, preferences, and linked entities such as orders or investments. Users are categorized as *Customer*, *Shareholder*, *Investor*, *Administrator*, *Community Member*, or *Delivery Partner*.
2. **Produce Listing** – Holds all product-related information such as produce ID, crop type, harvest date, price, quantity, origin farm, and sustainability metrics. It is directly linked to farmer and shareholder accounts.
3. **Order Record** – Captures purchase details including order ID, customer ID, ordered items, delivery partner, payment method, and delivery status. Orders interact dynamically with inventory and logistics modules.
4. **Investment Portfolio** – Manages investor contributions to ongoing farm projects. Each record includes investment ID, project name, funding amount, expected returns, and sustainability impact score.

5. **Entitlement Registry** – Tracks shareholder allocations and produce claims. It includes entitlement ID, shareholder ID, product type, distribution date, and delivery status.
6. **AI Modules Data** – Internal AI containers store processed data for demand forecasting, stock optimization, recommendations, and resource analytics.
7. **Community Engagement Record** – Maintains logs of member participation in workshops, blog posts, and community events.
8. **Support Tickets and Reports** – Contain complaint IDs, sender information, timestamps, issue type, status, and administrator response.
9. **Audit Logs** – Track every major user and admin action for transparency and traceability.
10. **Loyalty Ledger** – Tracks earned reward points for eco-friendly or community actions, enabling redemption in future purchases.

2.2 Global Data Structure:

The global data architecture provides centralized access for the entire system, integrating all operational modules under a unified database network.

Components:

- **Cloud SQL Database:** Core transactional data (users, orders, payments, inventory).
- **AI Analytics Warehouse:** Predictive models, aggregated forecasts, and trend data.
- **Document Storage:** Holds product images, certificates, and event resources.
- **API Integration Layer:** Handles external data from payment gateways, logistics partners, and agricultural datasets.
- **Backup and Recovery Store:** Nightly replication to ensure zero data loss.

Global structures use unique keys for all entities and enforce referential integrity across tables such as User, Product, Order, Investment, and Delivery.

2.3 Temporary Data Structure:

Temporary data elements are transient in nature and exist only for quick computation, caching, or buffering during active sessions.

Temporary Structure	Purpose
Session Cache	Stores active user session tokens, preferences, and temporary login states.
Search Cache	Retains recent search queries for faster re-access.
Cart Buffer	Maintains selected items before checkout; auto-expires after inactivity.
Notification Queue	Queues payment confirmations, stock alerts, and order updates.
Message Buffer	Temporarily holds messages exchanged during real-time chats.
AI Inference Cache	Stores intermediate results from forecasting models.
Delivery Dispatch Queue	Temporarily manages assignment of deliveries awaiting driver acceptance.
Event Feedback Queue	Collects workshop feedback before aggregation.

These structures are periodically flushed to maintain optimal performance and low memory consumption.

2.4 Database Description

The **Shared Farm Database Model** combines **relational data storage** for transactions and **document storage** for multimedia and AI outputs. It ensures ACID compliance, referential integrity, and secure multi-user access.

2.4.1 Core Database (Tables and Descriptions)

Table Name: User

Attribute	Data Type	Constraint
userID	string	Primary Key
name	string	
email	string	Unique
passwordHash	string	
phoneNumber	string	

role	enum(Customer, Shareholder, Investor, Admin, DeliveryPartner, CommunityMember)	
registrationDate	date	
accountStatus	string	Default: Active
address	string	
rewardPoints	int	Default: 0
lastLogin	date	

Description:

Stores all registered users and their access roles; forms the parent reference for all dependent tables.

Table Name: Product

Attribute	Data Type	Constraint
productID	string	Primary Key
farmerID	string	Foreign Key(User.userID)
name	string	
category	string	
description	string	
harvestDate	date	
price	double	
stock	int	
certification	string	
sustainabilityScore	double	
imageURL	string	

Description:

Represents produce listings available in the marketplace, each linked to its originating farmer.

Table Name: Order

Attribute	Data Type	Constraint
orderID	string	Primary Key
customerID	string	Foreign Key(User.userID)
productID	string	Foreign Key(Product.productID)

InnovaCode

quantity	int	
totalAmount	double	
paymentID	string	Foreign Key(Payment.paymentID)
deliveryPartnerID	string	Foreign Key(User.userID)
orderDate	date	
deliveryStatus	string	Default: Pending
trackingCode	string	

Description:

Records every transaction from order placement to delivery completion.

Table Name: Investment

Attribute	Data Type	Constraint
investmentID	string	Primary Key
investorID	string	Foreign Key(User.userID)
projectName	string	
fundingGoal	double	
contribution	double	
expectedReturn	double	
status	string	
startDate	date	
maturityDate	date	

Description:

Tracks investor contributions, expected ROI, and funding progress.

Table Name: Entitlement

Attribute	Data Type	Constraint
entitlementID	string	Primary Key
shareholderID	string	Foreign Key(User.userID)
productID	string	Foreign Key(Product.productID)
quantity	int	
issueDate	date	
deliveryStatus	string	

Description:

Logs shareholder produce allocations and monitors their delivery or exchange status.

Table Name: Payment

Attribute	Data Type	Constraint
paymentID	string	Primary Key
orderID	string	Foreign Key(Order.orderID)
payerID	string	Foreign Key(User.userID)
amount	double	
method	string	
paymentDate	date	
status	string	
refundStatus	string	Default: None
referenceNumber	string	

Description:

Captures transaction details, ensuring secure recording of financial activities.

Table Name: Delivery

Attribute	Data Type	Constraint
deliveryID	string	Primary Key
orderID	string	Foreign Key(Order.orderID)
deliveryPartnerID	string	Foreign Key(User.userID)
pickupTime	date	
deliveryTime	date	
status	string	Default: Scheduled
route	string	
feedback	string	

Description:

Stores logistics details for each order's delivery process.

Table Name: SustainabilityMetric

Attribute	Data Type	Constraint
-----------	-----------	------------

metricID	string	Primary Key
farmID	string	Foreign Key(User.userID)
waterUsage	double	
fertilizerUsage	double	
energyConsumption	double	
carbonSaved	double	
reportDate	date	

Description:

Holds sustainability KPIs and supports the AI modules in evaluating performance.

Table Name: CommunityEvent

Attribute	Data Type	Constraint
eventID	string	Primary Key
eventName	string	
eventDate	date	
location	string	
organizerID	string	Foreign Key(User.userID)
description	string	
maxParticipants	int	
registeredUsers	int	

Description:

Represents training sessions, workshops, and community outreach activities.

Table Name: SupportTicket

Attribute	Data Type	Constraint
ticketID	string	Primary Key
userID	string	Foreign Key(User.userID)
issueType	string	
description	string	
status	string	
assignedAgent	string	
createdAt	date	

resolvedAt	date	
------------	------	--

Description:

Records support requests and their resolution history.

Table Name: AuditLog

Attribute	Data Type	Constraint
logID	string	Primary Key
userID	string	Foreign Key(User.userID)
actionType	string	
actionDetail	string	
timestamp	date	
ipAddress	string	

Description:

Captures all significant system actions for accountability and traceability.

2.4.2 AI and Analytics Database

Table Name: DemandForecast

Attribute	Data Type	Constraint
forecastID	string	Primary Key
productID	string	Foreign Key(Product.productID)
forecastDate	date	
predictedDemand	double	
confidence	double	
modelVersion	string	
generatedBy	string	

Description:

AI-based forecast records used to predict upcoming demand.

Table Name: RecommendationRecord

Attribute	Data Type	Constraint
recordID	string	Primary Key

InnovaCode

userID	string	Foreign Key(User.userID)
recommendedItems	string	
generatedAt	date	
rating	double	
feedback	string	

Description:

Logs AI-generated recommendations, recipes, and storage tips for users.

Table Name: ResourceOptimization

Attribute	Data Type	Constraint
optimizationID	string	Primary Key
farmID	string	Foreign Key(User.userID)
parameter	string	
suggestedAction	string	
expectedGain	double	
generatedAt	date	

Description:

Contains AI-suggested improvements for resource use (water, fertilizer, energy).

Table Name: AlertQueue

Attribute	Data Type	Constraint
alertID	string	Primary Key
productID	string	Foreign Key(Product.productID)
alertType	string	
message	string	
createdAt	date	
status	string	Default: Pending

Description:

Stores automated alerts triggered by stock, delivery, or payment events.

InnovaCode

Table Name: LoyaltyLedger

Attribute	Data Type	Constraint
entryID	string	Primary Key
userID	string	Foreign Key(User.userID)
activityType	string	
pointsEarned	int	
date	date	
redeemed	bool	

Description:

Manages reward points and eco-action credits for users.

Section 3 – Architectural and Component-Level Design:

3.0 Architectural and Component-Level Design:

Suitable Architecture Style: Layered Architecture Style

Reasoning:

Layered architecture is the optimal choice for the Shared Farm System due to several critical factors. This architectural pattern allows clear separation of concerns across different functional layers, which benefits both design and implementation phases. In an agricultural marketplace that integrates multiple stakeholders, AI-driven analytics, payment processing, and real-time logistics, a layered approach provides the modularity needed to manage complexity effectively.

The layered design offers easy maintenance and modification capabilities. As agricultural technology, payment systems, and AI models evolve rapidly, the ability to update individual layers without disrupting the entire system is invaluable. For instance, upgrading the AI forecasting module or integrating a new payment gateway can be accomplished within specific layers without affecting user interfaces or database structures.

Additionally, layered architecture provides excellent scalability—a crucial requirement as the platform expands to accommodate more farms, regions, investors, and community members. Each layer can be scaled independently based on demand, whether it's adding more database replicas for data storage or enhancing processing power for AI computations.

This architecture also enhances security by establishing clear boundaries between user-facing components and sensitive data storage, allowing for targeted security measures at each layer. The separation of presentation, business logic, and data management aligns perfectly with the system's need for role-based access control and secure financial transactions.

Our layered architecture consists of 4 layers:

Presentation Layer (Layer 3)

Description:

The Presentation Layer is the topmost layer and serves as the primary interface between users and the Shared Farm System. This layer is responsible for capturing all user inputs and displaying system outputs in an intuitive, responsive format accessible across desktop browsers and mobile devices.

Responsibilities:

- **User Interface Rendering:** Displays dashboards customized for each user role (Customer, Shareholder, Investor, Administrator, Delivery Partner, Community Member)
- **Input Collection:** Captures user actions including registration, login credentials, product searches, order placements, investment selections, and event registrations
- **Data Visualization:** Presents sustainability metrics, financial analytics, demand forecasts, and delivery tracking through charts, graphs, and interactive maps
- **Navigation Control:** Manages page routing, menu systems, and user flow throughout the application
- **Responsive Design:** Ensures optimal display and functionality across various screen sizes and devices
- **Session Management:** Maintains active user sessions and handles timeouts

Key Components:

- Customer Marketplace Interface
- Shareholder Entitlement Portal
- Investor Dashboard
- Administrator Control Panel
- Delivery Partner Logistics Interface
- Community Engagement Hub
- Authentication Forms

- Notification Display System

Data Flow:

All user inputs collected in this layer are passed down to the Application Layer for validation and processing. Conversely, processed information, analytics, and system responses are received from the Application Layer and displayed to users in readable, visual formats.

Application Layer (Layer 2)

Description:

The Application Layer contains all core business logic and functional processing capabilities of the Shared Farm System. This layer acts as the intelligence hub, processing requests from the Presentation Layer, executing business rules, coordinating workflows, and managing communication between the user interface and data storage.

Responsibilities:

- **User Authentication & Authorization:** Validates login credentials, manages role-based permissions, and controls access to features
- **Order Processing:** Handles shopping cart operations, calculates totals, applies discounts, and manages checkout workflows
- **Payment Validation:** Interfaces with payment gateways, verifies transactions, and processes refund requests
- **Entitlement Management:** Calculates shareholder allocations, validates claims, and manages produce distribution
- **Investment Processing:** Evaluates project eligibility, processes funding contributions, and tracks returns
- **AI Module Coordination:** Triggers demand forecasting, stock automation, recommendation generation, and resource optimization
- **Delivery Scheduling:** Coordinates logistics, assigns delivery partners, and updates tracking status
- **Data Validation:** Ensures all user-submitted information meets quality and format requirements

- **Community Event Management:** Handles event registrations, participant tracking, and feedback collection
- **Notification Generation:** Creates alerts for stock levels, payment confirmations, delivery updates, and system events
- **Report Generation:** Compiles analytics, sustainability reports, and administrative summaries
- **Business Rule Enforcement:** Implements pricing logic, inventory rules, and transaction policies

Key Components:

- User Account Management Module
- Product Catalog & Search Engine
- Order Management System
- Payment Processing Interface
- Shareholder Entitlement Engine
- Investment Management Module
- AI Integration Controller
- Logistics Coordination System
- Community Event Coordinator
- Administrative Control Module
- Support Ticket Handler
- Reward & Loyalty Calculator

Data Flow:

This layer receives validated user inputs from the Presentation Layer, applies business logic and computations, then either retrieves information from or stores information to the Data & Analytics Layer. Processed results and system responses are sent back up to the Presentation Layer for user display.

Data & Analytics Layer (Layer 1)

Description:

The Data & Analytics Layer serves as the intelligent storage and processing backbone of the Shared Farm System. This layer is divided into two parallel subsystems: the Core Database System for transactional and operational data, and the AI Analytics Engine for predictive modeling and intelligent recommendations.

Responsibilities:

Core Database System:

- **Persistent Data Storage:** Stores all user profiles, product listings, orders, investments, entitlements, payments, deliveries, and audit logs
- **Data Integrity Management:** Enforces referential integrity, constraints, and ACID compliance across all transactions
- **Query Processing:** Executes complex database queries for retrieval, updates, insertions, and deletions
- **Backup & Recovery:** Performs nightly replications and maintains recovery points
- **Concurrency Control:** Manages simultaneous access from multiple users and processes

AI Analytics Engine:

- **Demand Forecasting:** Analyzes historical purchase patterns and seasonal trends to predict future demand
- **Stock Optimization:** Monitors inventory levels in real-time and triggers automated restock alerts
- **Personalized Recommendations:** Generates product suggestions, recipes, and storage tips based on user behavior
- **Resource Optimization:** Evaluates farm sustainability metrics and recommends efficiency improvements
- **Trend Analysis:** Identifies patterns in user behavior, sales, and farm performance
- **Model Training & Refinement:** Continuously improves AI accuracy using new data

Key Components:

Database Components:

- User Table (profiles, credentials, roles)

- Product Table (produce listings, pricing, stock)
- Order Table (transactions, delivery status)
- Investment Table (projects, contributions, returns)
- Entitlement Table (shareholder allocations)
- Payment Table (financial transactions)
- Delivery Table (logistics details)
- Sustainability Metrics Table (environmental KPIs)
- Community Event Table (workshops, registrations)
- Support Ticket Table (user assistance)
- Audit Log Table (system actions)

AI Components:

- Demand Forecast Module
- Stock Automation Engine
- Recommendation Generator
- Resource Optimization Analyzer
- Alert Queue Manager
- Loyalty Points Calculator

Data Flow:

The Application Layer sends queries, data insertion requests, and AI computation triggers to this layer. The Core Database System responds with requested records or confirmation of data operations. The AI Analytics Engine processes data patterns and returns forecasts, recommendations, and optimization suggestions back to the Application Layer.

Integration & Infrastructure Layer (Layer 0)

Description:

The Integration & Infrastructure Layer is the foundational layer that manages external connections, system infrastructure, and technical services that support all upper layers. This layer ensures the Shared Farm System can communicate with third-party services and maintain operational stability.

Responsibilities:

- **External API Integration:** Manages connections to payment gateways, logistics partners, weather data sources, and environmental databases
- **Cloud Infrastructure Management:** Oversees cloud server resources, load balancing, and scaling operations
- **Security Services:** Implements encryption, SSL certificates, firewall rules, and intrusion detection
- **Network Communication:** Handles HTTP/HTTPS protocols, WebSocket connections for real-time updates
- **Cache Management:** Operates temporary data structures for sessions, search queries, and notifications
- **Monitoring & Logging:** Tracks system performance, uptime, error rates, and resource utilization
- **Email & SMS Services:** Sends notification emails and text messages for alerts and confirmations
- **File Storage Services:** Manages document storage for product images, certificates, and event materials

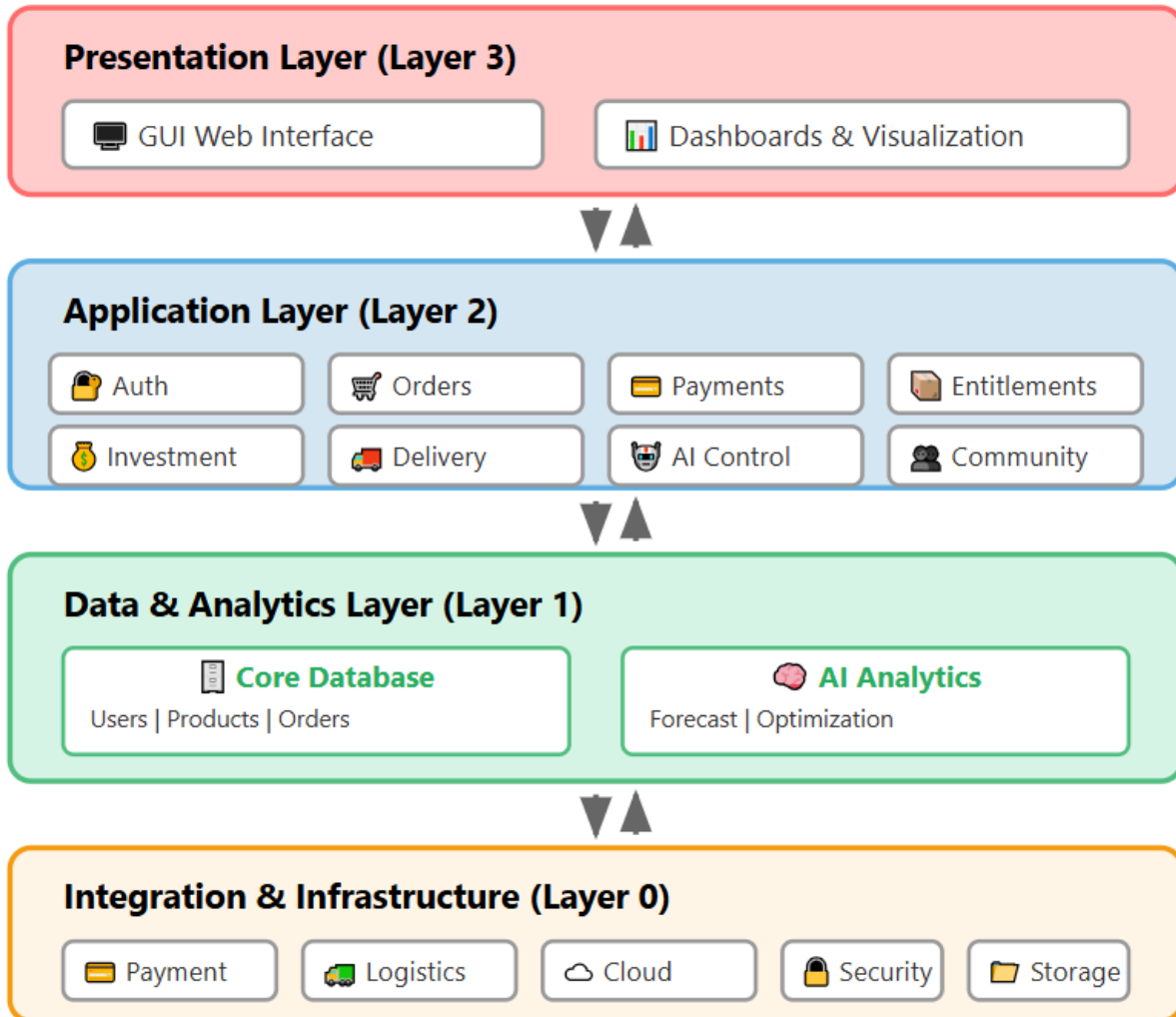
Key Components:

- Payment Gateway Connectors (Stripe, PayPal, local payment processors)
- Logistics API Integration (delivery routing, tracking services)
- Environmental Data APIs (weather, soil conditions, sustainability databases)
- Cloud Service Provider Interface (AWS, Azure, Google Cloud)
- Email Service Provider (SendGrid, AWS SES)
- SMS Gateway Service
- Content Delivery Network (CDN) for images and media
- Security & Firewall Systems
- Backup Storage Systems
- Session Cache Storage (Redis, Memcached)
- Notification Queue Manager

Data Flow:

This layer receives external service requests from the Application Layer (such as processing a payment or scheduling a delivery) and communicates with third-party APIs to fulfill these requests. Responses from external services are validated and passed back to the Application Layer.

Infrastructure services continuously support all layers by providing networking, caching, security, and monitoring capabilities.



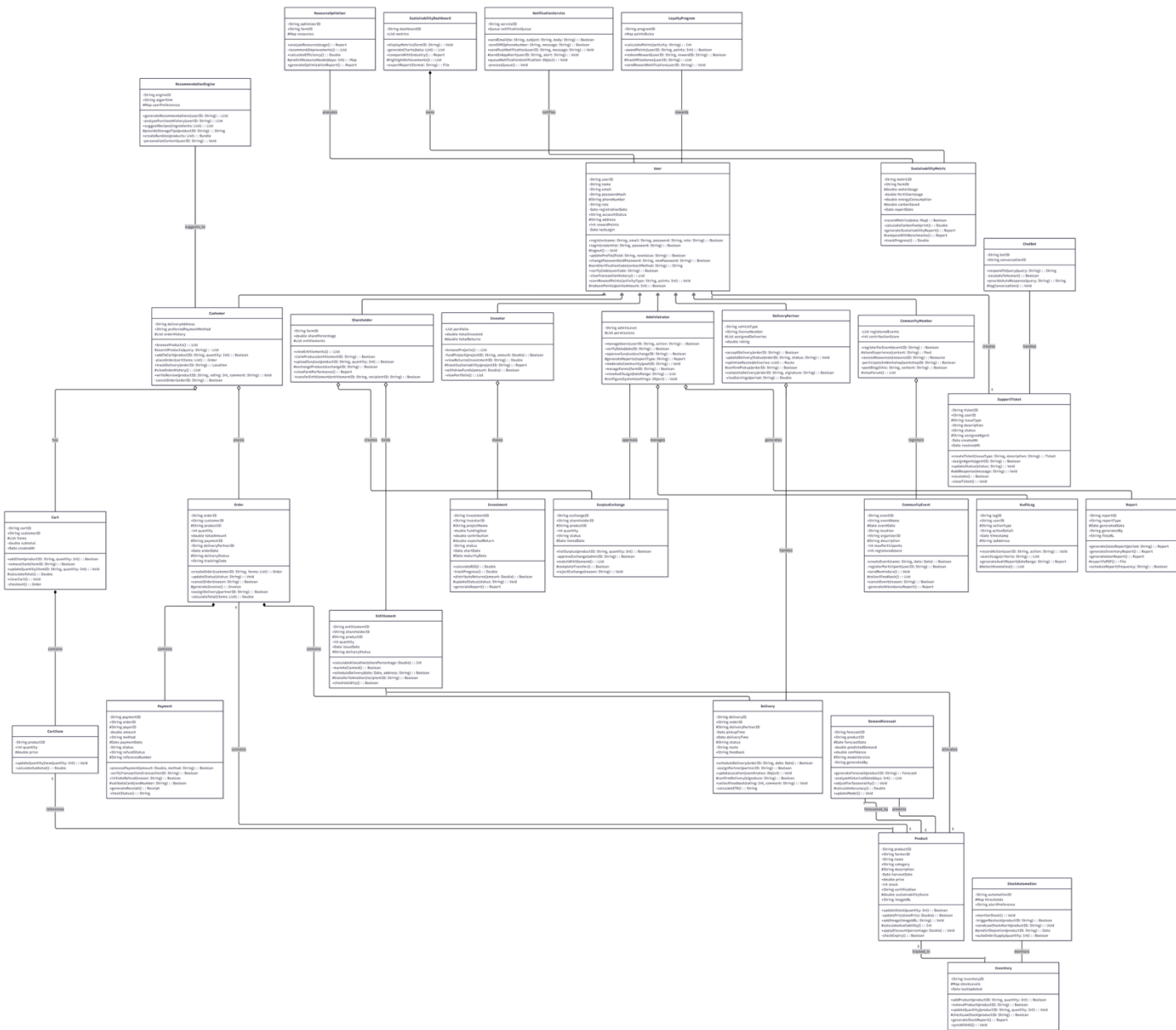
3.1 System Structure:

3.1.1 Architecture Diagram:

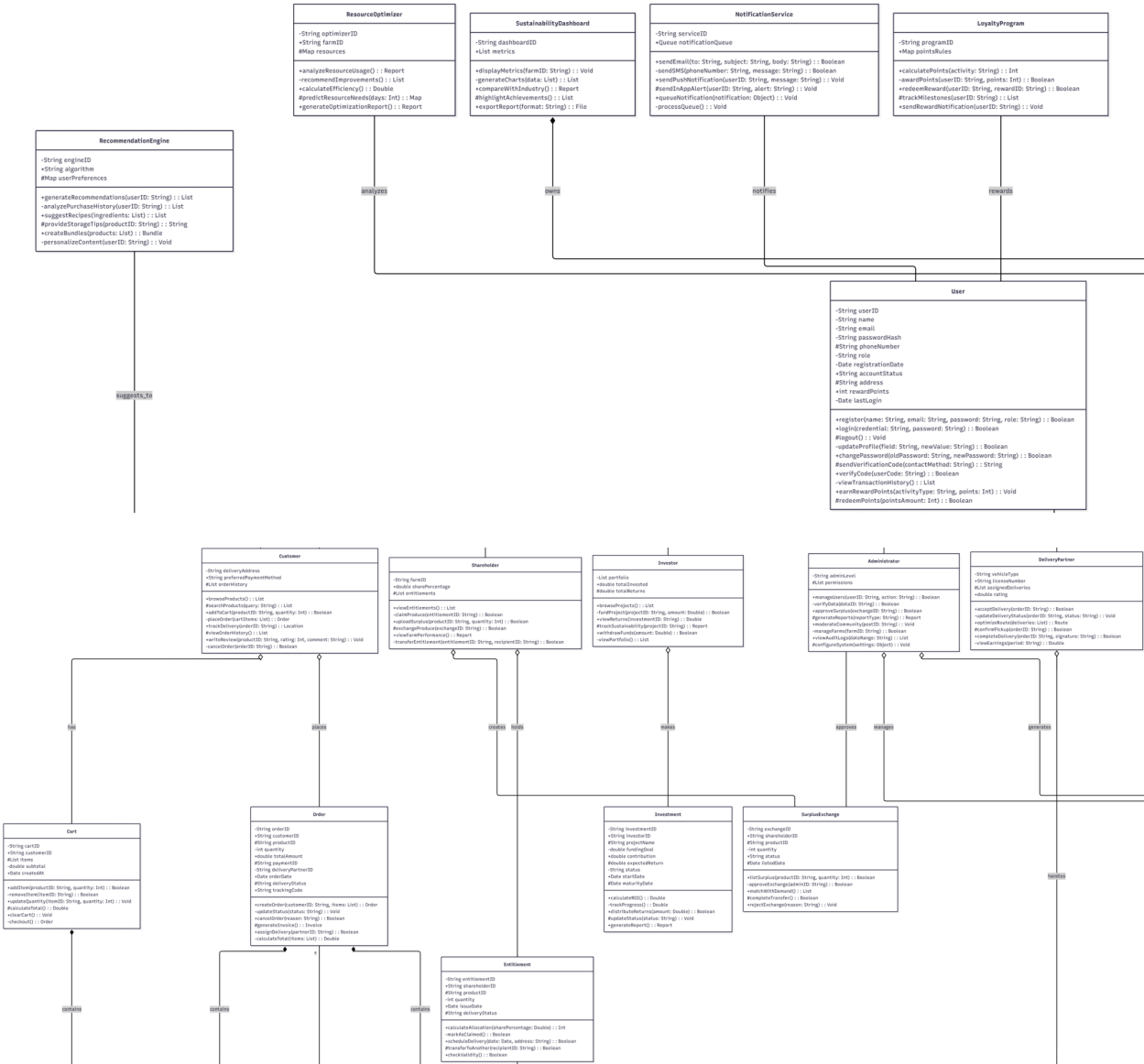
The following class diagram illustrates the complete object-oriented structure of the Shared Farm System, showing all major classes, their attributes, methods, and relationships. The diagram is organized to demonstrate the interactions between users, products, transactions, AI modules, and administrative components.

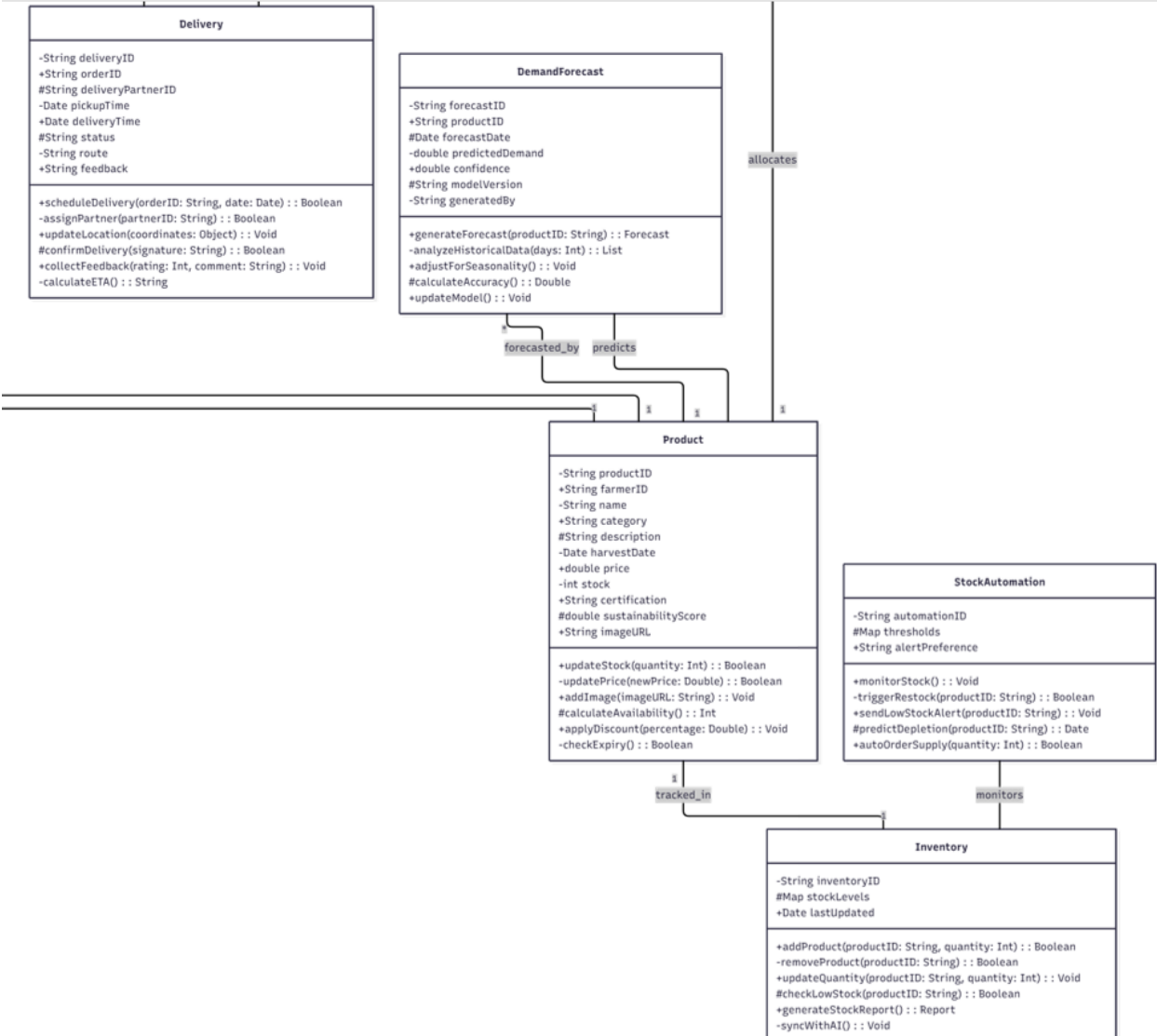
Diagram Overview:

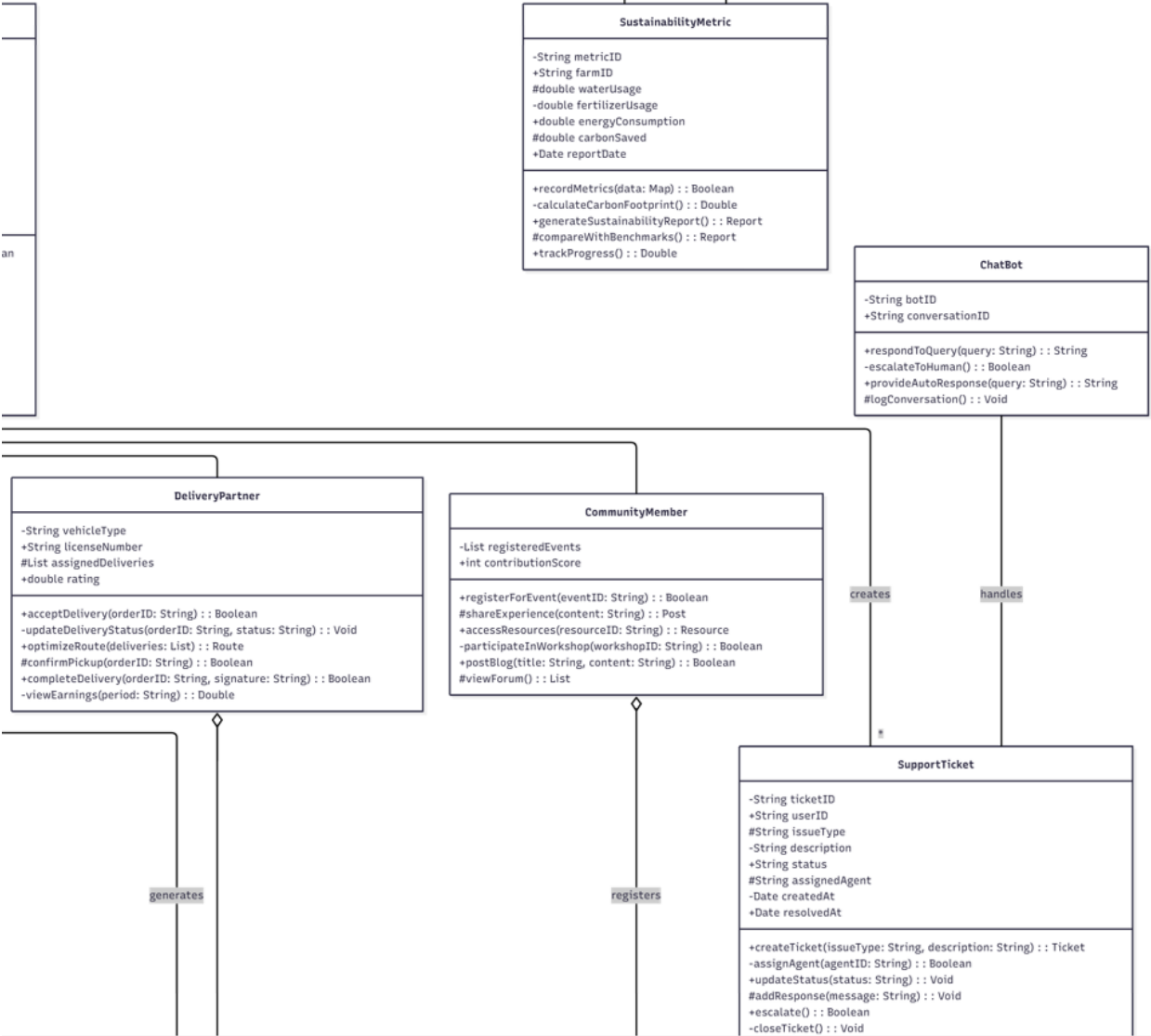
- **User Management Classes:** Handle authentication, roles, and profiles
- **Product & Inventory Classes:** Manage farm produce listings and stock
- **Transaction Classes:** Process orders, payments, and deliveries
- **Shareholder & Investment Classes:** Manage entitlements and funding
- **AI & Analytics Classes:** Provide intelligent forecasting and recommendations
- **Community & Support Classes:** Enable engagement and assistance
- **Administrative Classes:** Control system operations and reporting



For clarity we made parts of the class diagram to be more visible







3.2 Description for Components

3.2 Description for Component User

3.2.1 Processing narrative (PSPEC) for component User

The User component manages authentication, profile management, and user interactions. It handles registration, login, password management, reward points, transaction history, and support ticket submission for all platform participants (Customers, Shareholders, Investors, Administrators, Delivery Partners, and Community Members).

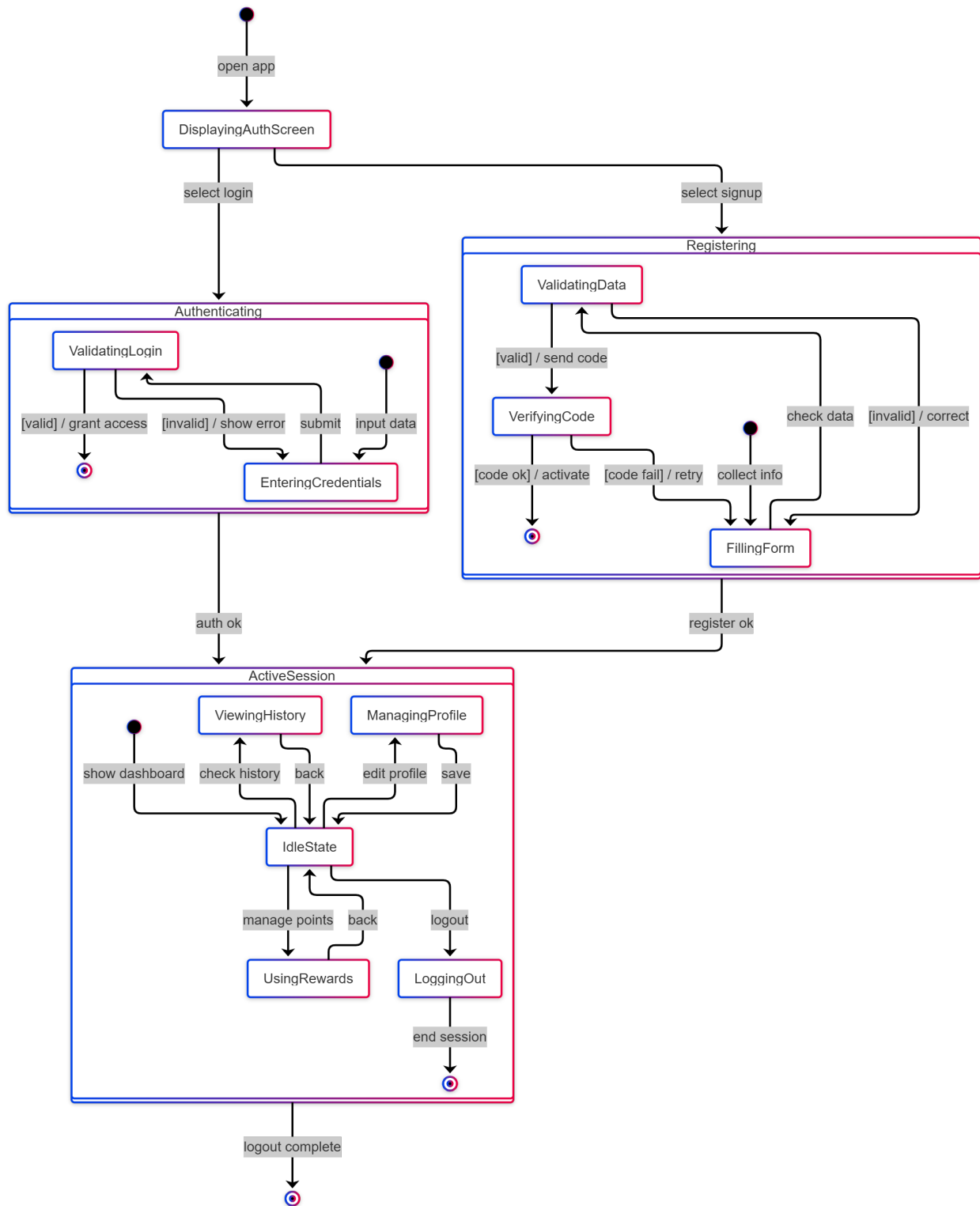
3.2.2 Component User interface description

The User interface appears when users open the application, requiring authentication before accessing features. It displays login/registration forms, verifies credentials through the database, and routes authenticated users to role-specific dashboards.

3.2.3 Component User processing detail

- Start
- User opens application
- User selects login or registration
- System validates credentials
- User accesses dashboard based on role
- User can view history, earn points, or submit tickets
- End

3.2.3.1 Design Class hierarchy for component User



3.2.3.2 Restrictions/limitations for component User

Users cannot access features without authentication. Email and phone verification required. Maximum 3 failed login attempts before temporary lock. Password reset limited to 3 verification attempts.

3.2.3.3 Performance issues for component User

Simultaneous login requests during peak hours may cause delays. Concurrent database queries for authentication may slow response time.

3.2.3.4 Design constraints for component User

Passwords must be 8+ characters with mixed case, numbers, and symbols. Session timeout after 30 minutes of inactivity. All authentication data encrypted in transit and at rest.

3.2.3.5 Processing detail for each operation of component User

1. register(name: String, email: String, password: String, role: String): Boolean
2. login(credential: String, password: String): Boolean
3. updateProfile(field: String, newValue: String): Boolean
4. changePassword(oldPassword: String, newPassword: String): Boolean
5. verifyCode(userCode: String): Boolean
6. earnRewardPoints(activityType: String, points: Int): Void
7. redeemPoints(pointsAmount: Int): Boolean
8. submitTicket(issueType: String, description: String): Ticket

3.2.3.5.1 Processing narrative (PSPEC) for each operation

1. **register()** - Creates new user account with role assignment and sends verification code
2. **login()** - Authenticates existing user and creates session token
3. **updateProfile()** - Modifies user information with validation
4. **changePassword()** - Updates password after verifying current password
5. **verifyCode()** - Validates verification code for account activation

6. **earnRewardPoints()** - Awards points for activities and updates loyalty ledger
7. **redeemPoints()** - Exchanges points for benefits if threshold met
8. **submitTicket()** - Creates support ticket and assigns to agent

3.2.3.5.2 Algorithmic model (PDL) for each operation

1. **register()**

Start

Input: name, email, password, role

Validate email format

IF email invalid THEN Return error

Check email exists in database

IF email exists THEN Return "Already registered"

Validate password strength

IF password weak THEN Return error

Hash password

Generate userID and set accountStatus = "Pending"

Save to database

Generate 6-digit code

Send code via email

Return success

End

2. **login()**

Start

Input: credential, password

InnovaCode

Query user by credential

IF user not found THEN Return "Invalid credentials"

Verify password against hash

IF incorrect THEN

Increment failed attempts

IF attempts $\geq$ 5 THEN Lock account

Return error

END IF

Generate session token

Update lastLogin timestamp

Return success with token

End

3. updateProfile()

Start

Input: field, newValue

Validate newValue format

IF invalid THEN Return error

IF field = "email" THEN

Check if email exists

IF exists THEN Return "Email in use"

Update email and set status = "Pending Verification"

ELSE

Update field directly

END IF

InnovaCode

Save to database

Return success

End

4. changePassword()

Start

Input: oldPassword, newPassword

Verify oldPassword

IF incorrect THEN Return "Wrong password"

Validate newPassword strength

IF weak THEN Return error

Hash newPassword

Update database

Send confirmation email

Return success

5. verifyCode()

Start

Input: userCode

Retrieve stored code and expiration

IF expired THEN Return "Code expired"

Compare codes

IF match THEN

Set accountStatus = "Active"

Return true

InnovaCode

ELSE

Return false

END IF

End

6. earnRewardPoints()

Start

Input: activityType, points

Retrieve current rewardPoints

Add points to total

Update database

Create LoyaltyLedger entry

IF total $\geq$ milestone THEN Send notification

Return

End

7. redeemPoints()

Start

Input: pointsAmount

Retrieve rewardPoints

IF points $<$ pointsAmount THEN Return "Insufficient points"

Subtract points

Update database

Generate discount code

Return success with code

InnovaCode

End

8. submitTicket()

Start

Input: issueType, description

Generate ticketID

Set status = "Open", createdAt = now

Assign to available agent

Save to database

Send notification to agent

Return ticket

End

3.2 Description for Component Order

3.2.1 Processing narrative (PSPEC) for component Order

The Order component manages purchase transactions from cart creation to delivery completion. It handles order placement, payment coordination, delivery assignment, status tracking, cancellations, and invoice generation.

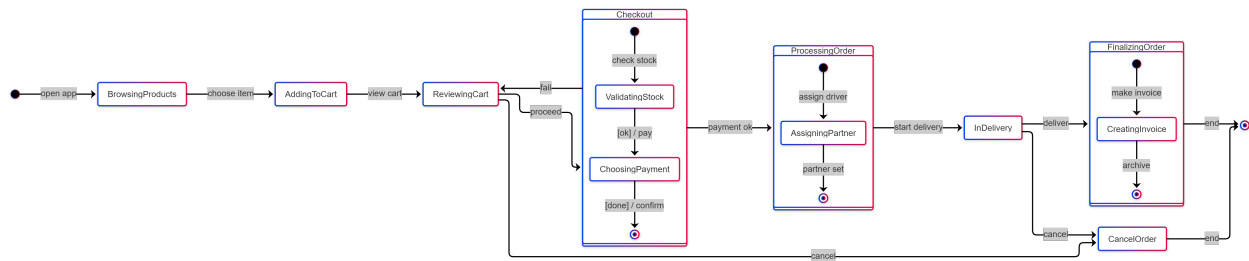
3.2.2 Component Order interface description

Displays when customers proceed to checkout. Shows cart summary, delivery options, payment methods, and order tracking. Updates real-time delivery status and allows order modifications within cancellation window.

3.2.3 Component Order processing detail

- Start
- Customer adds products to cart
- Customer proceeds to checkout
- System validates availability and calculates total
- Customer completes payment
- System creates order and assigns delivery partner
- Order progresses through delivery stages
- Customer confirms receipt
- End

3.2.3.1 Design Class hierarchy for component Order



3.2.3.2 Restrictions/limitations for component Order

Orders cannot be placed with out-of-stock items. Cancellation allowed only within 2 hours of placement. Delivery address cannot be changed after partner assignment. Minimum order value of \$10 required.

3.2.3.3 Performance issues for component Order

Multiple simultaneous orders on same product may cause stock inconsistency. Peak hour order processing may delay delivery partner assignment. Large order volumes may slow invoice generation.

3.2.3.4 Design constraints for component Order

Each order limited to 50 items maximum. Delivery slots limited to available time windows. Order tracking updates every 5 minutes during transit. Payment timeout after 10 minutes of inactivity.

3.2.3.5 Processing detail for each operation of component Order

1. `createOrder(customerID: String, items: List, deliveryAddress: String): Order`
2. `calculateTotal(items: List): Double`
3. `updateStatus(orderID: String, newStatus: String): Void`
4. `cancelOrder(orderID: String, reason: String): Boolean`
5. `assignDeliveryPartner(orderID: String): DeliveryPartner`
6. `generateInvoice(orderID: String): Invoice`
7. `trackOrder(orderID: String): Location`
8. `confirmDelivery(orderID: String, signature: String): Boolean`

3.2.3.5.1 Processing narrative (PSPEC) for each operation

1. **createOrder()** - Generates new order after payment confirmation and deducts stock
2. **calculateTotal()** - Computes total price including taxes and delivery fees
3. **updateStatus()** - Changes order delivery status and notifies customer
4. **cancelOrder()** - Processes cancellation if within allowed window and initiates refund
5. **assignDeliveryPartner()** - Finds available partner and sends delivery request
6. **generateInvoice()** - Creates detailed receipt with itemized charges
7. **trackOrder()** - Returns current location and estimated delivery time
8. **confirmDelivery()** - Marks order complete upon customer confirmation

3.2.3.5.2 Algorithmic model (PDL) for each operation

1. **createOrder()**

Start

Input: customerID, items, deliveryAddress

Validate items availability

IF any item unavailable THEN Return error

Generate unique orderID

Set orderDate = now, status = "Pending"

Generate trackingCode

FOR each item IN items DO

 Deduct quantity from stock

END FOR

Save order to database

Notify farmer

Return order

End

InnovaCode

2. calculateTotal()

Start

Input: items

Initialize total = 0

FOR each item IN items DO

total = total + (item.price * item.quantity)

END FOR

Calculate tax = total * 0.08

Calculate deliveryFee based on distance

total = total + tax + deliveryFee

Return total

End

3. updateStatus()

Start

Input: orderID, newStatus

Retrieve order from database

Update deliveryStatus = newStatus

Update timestamp

Save to database

Send notification to customer

Log in AuditLog

Return

End

4. cancelOrder()

Start

Input: orderID, reason

Retrieve order

Calculate time since orderDate

IF time > 2 hours THEN Return "Cannot cancel"

IF status = "InTransit" THEN Return "Too late to cancel"

Set status = "Canceled"

FOR each item IN order DO

 Restore stock quantity

END FOR

Initiate refund process

Update database

Return true

End

5. assignDeliveryPartner()

Start

Input: orderID

Retrieve order details

Query available delivery partners near deliveryAddress

IF no partners available THEN

 Set order status = "Delayed"

 Return null

END IF

InnovaCode

Select partner with highest rating

Send delivery request to partner

Wait for acceptance

IF accepted THEN

 Update order with deliveryPartnerID

 Return partner

ELSE

 Find next available partner

 Repeat process

END IF

End

6. generateInvoice()

Start

Input: orderID

Retrieve order details

Retrieve customer information

Retrieve product details

Create invoice document with:

- Order ID and date
- Customer details
- Itemized product list
- Subtotal, tax, delivery fee
- Total amount
- Payment method

InnovaCode

Generate PDF

Store in document storage

Return invoice

End

7. trackOrder()

Start

Input: orderID

Retrieve order with trackingCode

IF status = "Pending" THEN

 Return "Order being prepared"

ELSE IF status = "InTransit" THEN

 Query delivery partner location

 Calculate ETA based on current location

 Return location and ETA

ELSE IF status = "Delivered" THEN

 Return "Order delivered"

END IF

End

8. confirmDelivery()

Start

Input: orderID, signature

Retrieve order

IF status != "AtDestination" THEN Return false

Update status = "Delivered"

InnovaCode

Store signature

Set deliveryTime = now

Update database

Trigger reward points calculation

Generate final invoice

Return true

End

3.2 Description for Component Payment

3.2.1 Processing narrative (PSPEC) for component Payment

The Payment component handles all financial transactions including payment processing, verification, refunds, and receipt generation. It integrates with external payment gateways while maintaining PCI DSS compliance and secure transaction records.

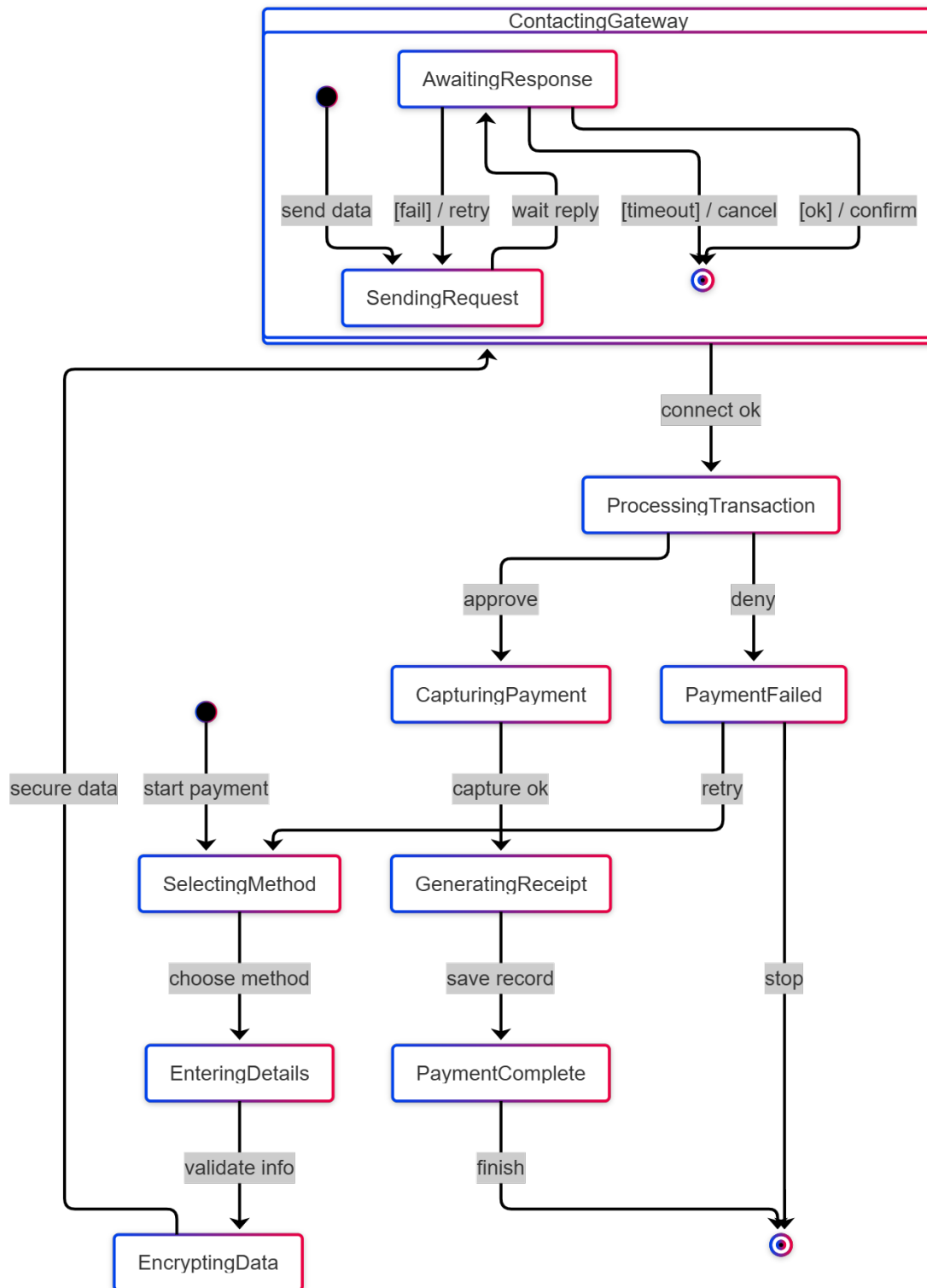
3.2.2 Component Payment interface description

Displays during checkout showing payment method options (card, wallet, bank transfer). Processes transactions securely through encrypted connections, shows real-time payment status, and generates digital receipts upon successful completion.

3.2.3 Component Payment processing detail

- Start
- Customer selects payment method
- System validates payment details
- Gateway processes transaction
- System confirms payment and generates receipt
- Payment record saved to database
- End

3.2.3.1 Design Class hierarchy for component Payment



3.2.3.2 Restrictions/limitations for component Payment

Only verified payment methods accepted. Transaction timeout after 10 minutes. Maximum 3 payment retry attempts. Refunds only processed for completed transactions. Card details not stored locally.

3.2.3.3 Performance issues for component Payment

Payment gateway timeouts during high traffic. Multiple simultaneous transactions may queue. Network latency affects authorization speed.

3.2.3.4 Design constraints for component Payment

All payment data encrypted using TLS 1.3. Tokenization used for card storage. Compliance with PCI DSS Level 1. Transaction logs immutable after creation. Receipts generated in PDF format only.

3.2.3.5 Processing detail for each operation of component Payment

1. `processPayment(orderID: String, amount: Double, method: String, details: Object): Payment`
2. `verifyTransaction(paymentID: String): Boolean`
3. `initiateRefund(paymentID: String, reason: String): Boolean`
4. `validateCard(cardNumber: String, cvv: String, expiry: String): Boolean`
5. `generateReceipt(paymentID: String): Receipt`
6. `checkStatus(paymentID: String): String`

3.2.3.5.1 Processing narrative (PSPEC) for each operation

1. **processPayment()** - Initiates payment through gateway and creates transaction record
2. **verifyTransaction()** - Confirms payment authorization and fund capture
3. **initiateRefund()** - Reverses completed payment and returns funds to customer
4. **validateCard()** - Checks card format, expiry date, and CVV validity
5. **generateReceipt()** - Creates detailed payment receipt with transaction details
6. **checkStatus()** - Returns current payment state (pending, completed, failed, refunded)

3.2.3.5.2 Algorithmic model (PDL) for each operation

1. processPayment()

Start

Input: orderID, amount, method, details

Generate unique paymentID

Set paymentDate = now, status = "Processing"

Encrypt payment details

Connect to payment gateway

IF connection failed THEN Return error

Send authorization request with amount

Wait for gateway response

IF authorized THEN

 Capture payment

 Set status = "Completed"

 Generate referenceNumber

 Create receipt

 Save to database

 Return payment

ELSE

 Set status = "Failed"

 Return error

END IF

End

2. verifyTransaction()

Start

Input: paymentID

Retrieve payment from database

IF status = "Completed" AND referenceNumber exists THEN

InnovaCode

Query gateway for confirmation

IF confirmed THEN Return true

END IF

Return false

End

3. initiateRefund()

Start

Input: paymentID, reason

Retrieve payment

IF status != "Completed" THEN Return "Cannot refund"

IF refundStatus = "Refunded" THEN Return "Already refunded"

Connect to payment gateway

Send refund request with paymentID and amount

IF refund approved THEN

Set refundStatus = "Refunded"

Update database

Send notification to customer

Return true

ELSE

Return false

END IF

End

4. validateCard()

Start

Input: cardNumber, cvv, expiry

Check cardNumber length = 16 digits

IF invalid THEN Return false

Validate Luhn algorithm on cardNumber

InnovaCode

IF fails THEN Return false

Parse expiry date

IF expiry < current date THEN Return false

Check cvv length = 3 or 4 digits

IF invalid THEN Return false

Return true

End

5. generateReceipt()

Start

Input: paymentID

Retrieve payment details

Retrieve order information

Retrieve customer details

Create receipt document with:

- Payment ID and date
- Order reference
- Amount paid
- Payment method (masked)
- Transaction reference

Generate PDF

Store in file system

Return receipt

End

6. checkStatus()

Start

Input: paymentID

Retrieve payment from database

Return status field

End

3.2 Description for Component Entitlement

3.2.1 Processing narrative (PSPEC) for component Entitlement

The Entitlement component manages shareholder produce allocations, claim processing, delivery scheduling, and surplus exchange. It calculates shares based on ownership percentage and tracks distribution status for transparent shareholder management.

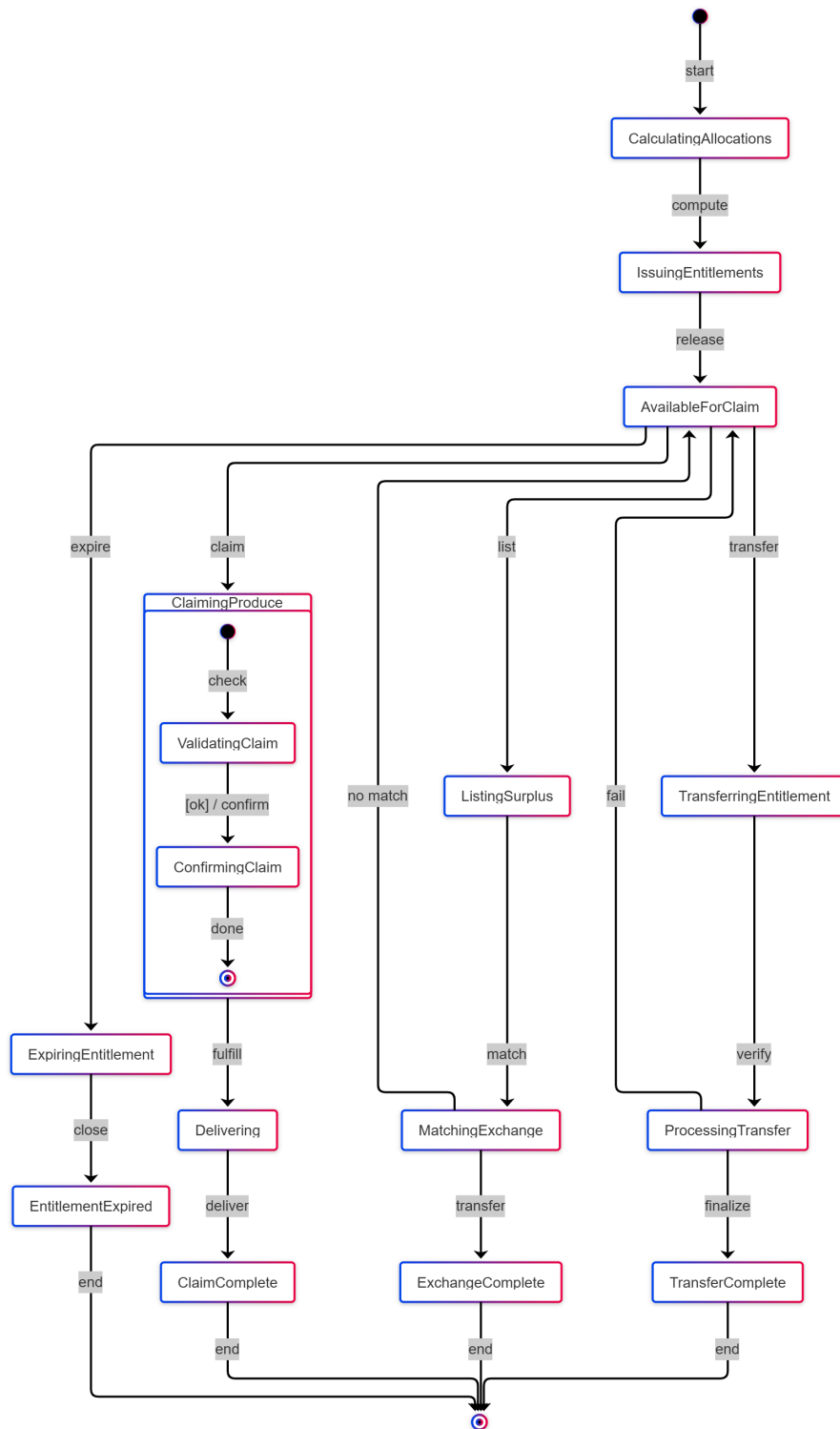
3.2.2 Component Entitlement interface description

Displays shareholder dashboard showing available entitlements, claim history, and upcoming distributions. Allows shareholders to claim produce, schedule deliveries, transfer entitlements, or list surplus for exchange with administrator approval.

3.2.3 Component Entitlement processing detail

- Start
- System calculates shareholder allocations
- Shareholder views available entitlements
- Shareholder claims produce or lists surplus
- System validates claim and updates inventory
- Delivery scheduled or exchange processed
- End

3.2.3.1 Design Class hierarchy for component Entitlement



3.2.3.2 Restrictions/limitations for component Entitlement

Entitlements expire 30 days after issue date. Only verified shareholders can claim. Transfer requires both parties' consent. Surplus listings need administrator approval. Entitlements cannot exceed shareholder's ownership percentage.

3.2.3.3 Performance issues for component Entitlement

Allocation calculations for large shareholder groups may take time. Simultaneous claims on limited stock may cause conflicts. Exchange matching algorithm slows with many listings.

3.2.3.4 Design constraints for component Entitlement

Entitlement calculations update weekly. Maximum 10 active entitlements per shareholder. Surplus must be listed within 7 days of harvest. Transfer fee of 2% applied. Delivery scheduling limited to available slots.

3.2.3.5 Processing detail for each operation of component Entitlement

1. `calculateAllocation(shareholderID: String, productID: String): Int`
2. `issueEntitlement(shareholderID: String, productID: String, quantity: Int): Entitlement`
3. `claimProduce(entitlementID: String, deliveryDate: Date): Boolean`
4. `scheduleDelivery(entitlementID: String, address: String, slot: String): Delivery`
5. `transferEntitlement(entitlementID: String, recipientID: String): Boolean`
6. `listSurplus(shareholderID: String, productID: String, quantity: Int): SurplusExchange`
7. `checkValidity(entitlementID: String): Boolean`

3.2.3.5.1 Processing narrative (PSPEC) for each operation

1. **calculateAllocation()** - Computes shareholder's produce quantity based on ownership percentage
2. **issueEntitlement()** - Creates entitlement record with issue date and delivery status
3. **claimProduce()** - Processes shareholder claim and marks entitlement as pending delivery
4. **scheduleDelivery()** - Assigns delivery slot and creates delivery record
5. **transferEntitlement()** - Reassigns entitlement to another shareholder with validation
6. **listSurplus()** - Creates surplus exchange listing pending admin approval

7. **checkValidity()** - Verifies entitlement is within claim period and not already claimed

3.2.3.5.2 Algorithmic model (PDL) for each operation

1. **calculateAllocation()**

Start

Input: shareholderID, productID

Retrieve shareholder ownership percentage

Retrieve total product harvest quantity

Calculate allocation = harvest * (percentage / 100)

Round down to whole units

Return allocation

End

2. **issueEntitlement()**

Start

Input: shareholderID, productID, quantity

Generate unique entitlementID

Set issueDate = current date

Set deliveryStatus = "Available"

Create Entitlement record

Save to database

Send notification to shareholder

Return entitlement

End

3. **claimProduce()**

Start

Input: entitlementID, deliveryDate

InnovaCode

Retrieve entitlement

Check deliveryStatus = "Available"

IF not available THEN Return false

Calculate days since issueDate

IF days > 30 THEN Return "Expired"

Check product stock availability

IF insufficient stock THEN Return "Out of stock"

Set deliveryStatus = "Claimed"

Update database

Return true

End

4. scheduleDelivery()

Start

Input: entitlementID, address, slot

Retrieve entitlement

Validate deliveryStatus = "Claimed"

IF not claimed THEN Return error

Check slot availability

IF slot unavailable THEN Return "Slot full"

Create Delivery record with:

- entitlementID

- deliveryAddress = address

- scheduledTime = slot

- status = "Scheduled"

Update entitlement deliveryStatus = "Scheduled"

Save to database

Return delivery

InnovaCode

End

5. transferEntitlement()

Start

Input: entitlementID, recipientID

Retrieve entitlement

Validate deliveryStatus = "Available"

IF not available THEN Return "Cannot transfer"

Verify recipient is valid shareholder

IF invalid THEN Return error

Calculate transfer fee = quantity * 0.02

Update shareholderID = recipientID

Log transfer in AuditLog

Send notifications

Return true

End

6. listSurplus()

Start

Input: shareholderID, productID, quantity

Generate exchangeID

Check shareholder has surplus quantity

IF insufficient THEN Return error

Set status = "Pending Approval"

Set listedDate = current date

Create SurplusExchange record

Save to database

Notify administrator for approval

InnovaCode

Return exchange

End

7. checkValidity()

Start

Input: entitlementID

Retrieve entitlement

Calculate days since issueDate

IF days > 30 THEN Return false

IF deliveryStatus = "Delivered" THEN Return false

Return true

End

3.2 Description for Component AI Stock Automation

3.2.1 Processing narrative (PSPEC) for component AI Stock Automation

The AI Stock Automation component continuously monitors inventory levels across all products in the Shared Farm System. It automatically detects low stock situations, generates alerts, and triggers restock processes. The component analyzes real-time stock data, applies predefined thresholds, and communicates with farmers and administrators to maintain optimal inventory levels and prevent stockouts.

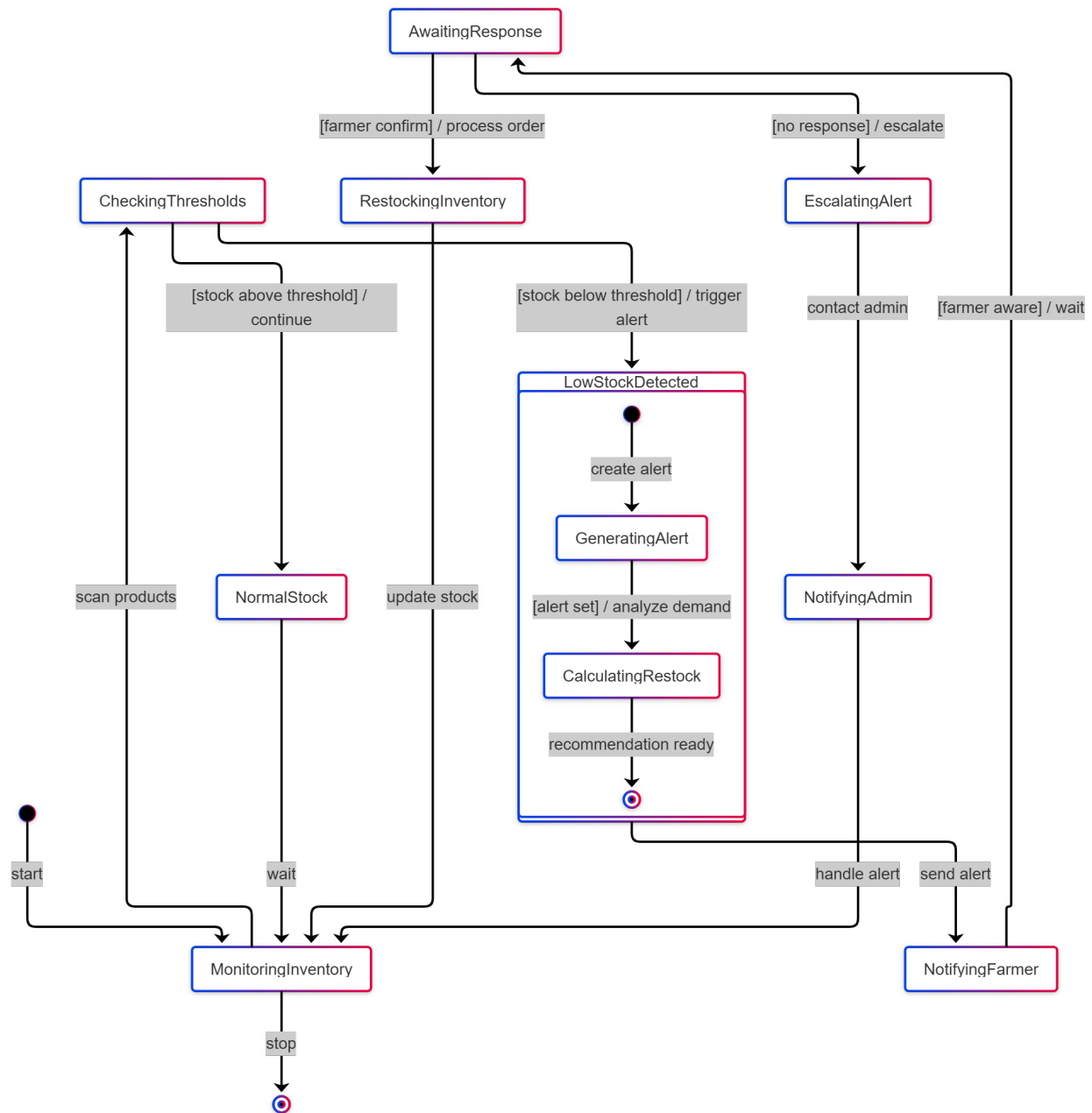
3.2.2 Component AI Stock Automation interface description

The AI Stock Automation interface operates in the background without direct user interaction. It displays automated alerts on farmer and administrator dashboards when stock levels fall below thresholds. The interface shows current stock levels, restock recommendations, and historical stock depletion patterns through visual indicators and notifications.

3.2.3 Component AI Stock Automation processing detail

- Start
- System monitors inventory levels continuously
- AI module checks stock against thresholds
- Low stock detected and alert generated
- System notifies farmer and administrator
- Restock request created automatically
- Farmer confirms or adjusts restock quantity
- Inventory updated after restocking
- End

3.2.3.1 Design Class hierarchy for component AI Stock Automation



3.2.3.2 Restrictions/limitations for component AI Stock Automation

Stock monitoring runs at 1-hour intervals minimum. Threshold adjustments require administrator approval. Cannot auto-restock without farmer confirmation. Alert escalation limited to 3 levels. Historical data must span at least 30 days for accurate threshold calculation.

3.2.3.3 Performance issues for component AI Stock Automation

Monitoring thousands of products simultaneously may cause processing delays. Multiple concurrent low-stock alerts can overwhelm the notification system. Real-time inventory updates during high-traffic periods may lag.

3.2.3.4 Design constraints for component AI Stock Automation

Thresholds must be between 5% and 50% of total stock. Alert frequency limited to one per product per day. Restock quantities calculated based on 7-day average demand. System requires 24/7 uptime for continuous monitoring.

3.2.3.5 Processing detail for each operation of component AI Stock Automation

1. `monitorInventory()`: Continuously scans all product stock levels
2. `checkThreshold(productID)`: Compares current stock against predefined limits
3. `generateAlert(productID, stockLevel)`: Creates low-stock notification
4. `calculateRestockQuantity(productID)`: Determines optimal reorder amount
5. `notifyStakeholder(alertData)`: Sends notifications to farmers and admins
6. `updateThreshold(productID, newThreshold)`: Adjusts stock limits
7. `escalateAlert(alertID)`: Forwards unresolved alerts to administrators
8. `confirmRestock(productID, quantity)`: Validates and processes restock order

3.2.3.5.1 Processing narrative (PSPEC) for each operation

1. **`monitorInventory()`** - Continuously scans database for current stock levels across all products
2. **`checkThreshold()`** - Compares product stock level against threshold percentage
3. **`generateAlert()`** - Creates alert record with product details and recommended restock quantity
4. **`calculateRestockQuantity()`** - Analyzes 7-day demand average to determine reorder amount
5. **`notifyStakeholder()`** - Sends email/SMS notifications to relevant farmers and administrators
6. **`updateThreshold()`** - Modifies stock threshold percentage after admin approval
7. **`escalateAlert()`** - Forwards unresolved alerts to higher authority after timeout
8. **`confirmRestock()`** - Processes farmer confirmation and updates inventory records

3.2.3.5.2 Algorithmic model (PDL) for each operation

1. monitorInventory()

Start

Query database for all products

FOR each product DO

Retrieve current stock level

Call checkThreshold(productID)

END FOR

Wait 1 hour

Repeat

End

2. checkThreshold()

Start

Input: productID

Retrieve product stock and threshold

Calculate percentage = $(\text{currentStock} / \text{maxStock}) * 100$

IF percentage $\leq$ threshold THEN

Call generateAlert(productID, currentStock)

END IF

End

3. generateAlert()

Start

Input: productID, stockLevel

Generate alertID

Set alertDate = current date

InnovaCode

Set priority based on stock level

Call calculateRestockQuantity(productID)

Create Alert record in database

Return alert

End

4. calculateRestockQuantity()

Start

Input: productID

Retrieve 7-day sales history

Calculate average daily demand

Calculate recommended quantity = average * 14 days

Return quantity

End

5. notifyStakeholder()

Start

Input: alertData

Retrieve farmer email and phone

Send email notification with alert details

Send SMS with alert summary

Log notification in database

Return success

End

6. updateThreshold()

Start

Input: productID, newThreshold

InnovaCode

Validate threshold range (5% to 50%)

IF invalid THEN Return error

Update product threshold in database

Log change in AuditLog

Return success

End

7. escalateAlert()

Start

Input: alertID

Retrieve alert details

Increment escalation level

Retrieve administrator contact

Send notification to admin

Update alert status in database

Return success

End

8. confirmRestock()

Start

Input: productID, quantity

Validate quantity > 0

Update stock in database

Mark alert as resolved

Notify farmer of completion

Return success

End

3.2 Description for Component Delivery Status Tracking

3.2.1 Processing narrative (PSPEC) for component Delivery Status Tracking

The Delivery Status Tracking component manages the complete lifecycle of order deliveries from assignment through completion. It coordinates with delivery partners to update location data, calculates ETAs, handles delivery exceptions, and manages customer notifications throughout the delivery process.

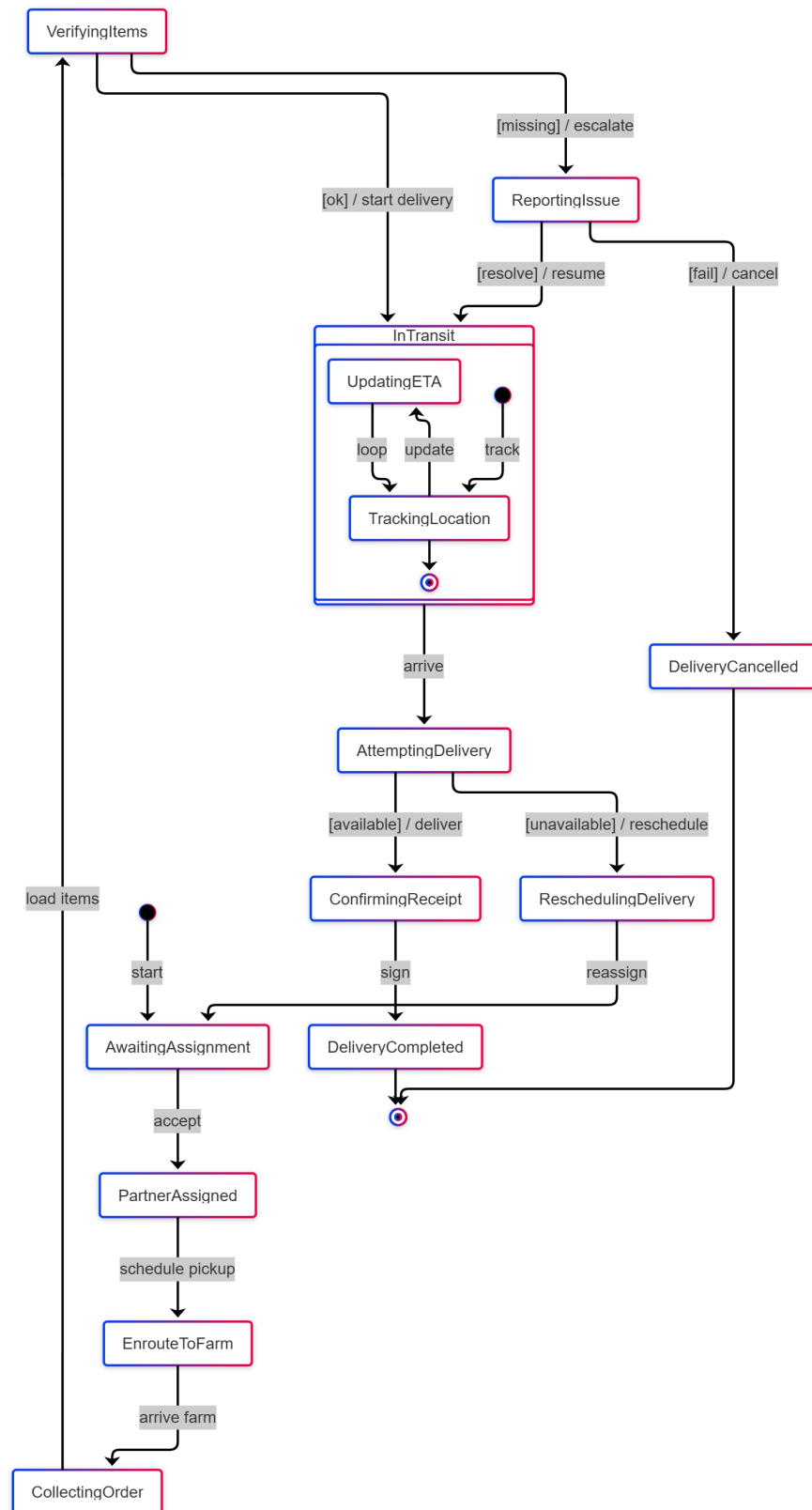
3.2.2 Component Delivery Status Tracking interface description

The interface displays real-time delivery progress to customers through interactive maps and status updates. Delivery partners use it to update their location and delivery status. The system shows estimated arrival times, current location markers, delivery route visualization, and allows customers to communicate with delivery partners.

3.2.3 Component Delivery Status Tracking processing detail

- Start
- Delivery partner assigned to order
- System tracks partner location continuously
- Status updated at each delivery milestone
- Customer receives real-time notifications
- Exceptions handled and escalated if needed
- Delivery confirmed upon completion
- End

3.2.3.1 Design Class hierarchy for component delivery status tracking



3.2.3.2 Restrictions/limitations for component Delivery Status Tracking

Location updates must occur every 5 minutes during transit. Maximum 3 delivery attempts before rescheduling required. ETA calculations accurate within 15-minute window. Cannot track deliveries without GPS-enabled devices.

3.2.3.3 Performance issues for component Delivery Status Tracking

High volume of simultaneous deliveries may slow location updates. GPS signal loss in rural areas affects tracking accuracy. Real-time ETA calculations resource-intensive during peak hours.

3.2.3.4 Design constraints for component Delivery Status Tracking

Delivery partners must have smartphones with GPS. Location data retained for 90 days only. Customer notification frequency limited to prevent spam. Signature capture required for orders above \$100.

3.2.3.5 Processing detail for each operation of component Delivery Status Tracking

1. `assignPartner(orderID, partnerID)`: Assigns delivery partner to order
2. `updateLocation(partnerID, coordinates)`: Records current GPS position
3. `calculateETA(currentLocation, destination)`: Computes estimated arrival time
4. `notifyCustomer(orderID, status)`: Sends delivery status updates
5. `confirmPickup(orderID)`: Marks order collected from farm
6. `reportIssue(orderID, issueType)`: Logs delivery problems
7. `confirmDelivery(orderID, signature)`: Finalizes successful delivery
8. `rescheduleDelivery(orderID, newDate)`: Sets new delivery slot

3.2.3.5.1 Processing narrative (PSPEC) for each operation

1. **assignPartner()** - Links delivery partner to order and notifies both parties
2. **updateLocation()** - Records GPS coordinates and updates tracking map
3. **calculateETA()** - Uses current location and traffic data to estimate arrival time
4. **notifyCustomer()** - Sends SMS/email updates at each delivery milestone

5. **confirmPickup()** - Verifies order collected from farm and starts transit tracking
6. **reportIssue()** - Documents delivery problems and escalates to support
7. **confirmDelivery()** - Records signature and marks order complete
8. **rescheduleDelivery()** - Sets new delivery date when customer unavailable

3.2.3.5.2 Algorithmic model (PDL) for each operation

1. **assignPartner()**

Start

Input: orderID, partnerID

Retrieve order details

Update order with deliveryPartnerID

Set status = "Assigned"

Notify partner with pickup details

Notify customer with partner info

Return success

End

2. **updateLocation()**

Start

Input: partnerID, coordinates

Retrieve current delivery order

Store coordinates with timestamp

Call calculateETA(coordinates, destination)

IF significant change THEN

 Call notifyCustomer(orderID, "Location updated")

END IF

Return success

End

3. calculateETA()

Start

Input: currentLocation, destination

Calculate distance between points

Retrieve current traffic data

Estimate travel time = distance / average_speed

Add buffer time (15 minutes)

Return ETA

End

4. notifyCustomer()

Start

Input: orderID, status

Retrieve customer contact info

Format notification message

Send SMS notification

Send email notification

Log notification in database

Return success

End

5. confirmPickup()

Start

Input: orderID

Update order status = "In Transit"

Record pickup timestamp

Notify customer "Order picked up"

Start location tracking

InnovaCode

Return success

End

6. reportIssue()

Start

Input: orderID, issueType

Generate issueID

Create issue record with details

Set order status = "Issue Reported"

Notify support team

Notify customer about delay

Return issueID

End

7. confirmDelivery()

Start

Input: orderID, signature

Store signature image

Update order status = "Delivered"

Record delivery timestamp

Stop location tracking

Notify customer "Delivered"

Calculate delivery partner rating

Return success

End

8. rescheduleDelivery()

Start

Input: orderID, newDate

Validate new date is future date

Update delivery schedule

Set status = "Rescheduled"

Release current partner

Find new available partner for date

Notify customer of new schedule

Return success

End

3.2 Description for Component AI Resource Optimizer

3.2.1 Processing narrative (PSPEC) for component AI Resource Optimizer

The AI Resource Optimizer analyzes farm resource utilization patterns including water, fertilizer, energy, and equipment usage. It identifies inefficiencies, generates optimization recommendations, and predicts resource needs based on historical data and upcoming harvest schedules to help farmers maximize efficiency and reduce waste.

3.2.2 Component AI Resource Optimizer interface description

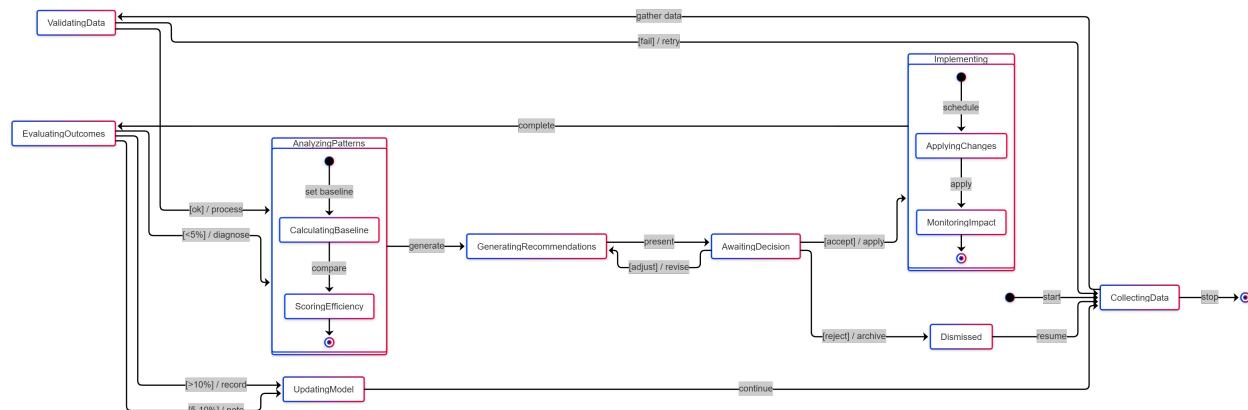
The interface presents resource usage analytics through dashboards showing consumption trends, efficiency scores, and improvement suggestions. Farmers view recommendations for water conservation, fertilizer reduction, energy optimization, and equipment scheduling. The system displays projected savings and environmental impact metrics.

3.2.3 Component AI Resource Optimizer processing detail

- Start
- System collects resource usage data from farms
- AI analyzes consumption patterns and efficiency
- Baseline metrics established and benchmarks compared

- Optimization opportunities identified
- Recommendations generated with projected savings
- Farmers review and implement suggestions
- System tracks improvement and adjusts recommendations
- End

3.2.3.1 Design Class hierarchy AI Resource Optimizer



3.2.3.2 Restrictions/limitations for component AI Resource Optimizer

Requires minimum 3 months of historical data for accurate analysis. Recommendations updated monthly only. Cannot optimize resources without IoT sensors installed. Maximum 5 active recommendations per farm at once.

3.2.3.3 Performance issues for component AI Resource Optimizer

Complex calculations for large farms may take several hours. Multiple concurrent optimization requests strain processing capacity. Real-time monitoring of implementations requires significant bandwidth.

3.2.3.4 Design constraints for component AI Resource Optimizer

Efficiency improvements must be measurable within 30 days. Recommendations prioritized by ROI and environmental impact. System requires integration with farm IoT devices. Savings calculations conservative with 20% margin.

3.2.3.5 Processing detail for each operation of component AI Resource Optimizer

1. `collectResourceData(farmID)`: Gathers water, fertilizer, energy, equipment usage
2. `validateData(resourceData)`: Checks data completeness and accuracy
3. `analyzePatterns(farmID)`: Identifies usage patterns and inefficiencies
4. `compareBenchmarks(farmID)`: Compares against industry standards
5. `generateRecommendations(farmID)`: Creates optimization suggestions
6. `calculateSavings(recommendationID)`: Estimates cost and environmental savings
7. `implementChanges(farmID, recommendationID)`: Applies approved optimizations
8. `trackImpact(implementationID)`: Monitors results and effectiveness

3.2.3.5.1 Processing narrative (PSPEC) for each operation

1. **`collectResourceData()`** - Retrieves usage data from IoT sensors and manual farm inputs
2. **`validateData()`** - Ensures data quality, completeness, and reasonable value ranges
3. **`analyzePatterns()`** - Uses ML algorithms to identify waste, inefficiencies, and optimization opportunities
4. **`compareBenchmarks()`** - Compares farm metrics with similar farms and industry standards
5. **`generateRecommendations()`** - Creates prioritized list of actionable improvements with expected benefits
6. **`calculateSavings()`** - Estimates financial savings and environmental impact of each recommendation
7. **`implementChanges()`** - Schedules and applies approved optimizations with farmer oversight
8. **`trackImpact()`** - Monitors resource usage changes and measures actual vs. projected improvements

3.2.3.5.2 Algorithmic model (PDL) for each operation

1. **`collectResourceData()`**

Start

Input: `farmID`

Query IoT sensors for water usage

Query IoT sensors for fertilizer usage

InnovaCode

Query IoT sensors for energy consumption

Query equipment usage logs

Aggregate all data by date

Return resourceData

End

2. validateData()

Start

Input: resourceData

Check all required fields present

IF missing fields THEN Return false

Validate value ranges (usage ≥ 0)

Check for outliers (values > 3 std deviations)

IF anomalies detected THEN Flag for review

Return true

End

3. analyzePatterns()

Start

Input: farmID

Retrieve 90 days of resource data

Calculate daily averages for each resource

Identify peak usage periods

Detect waste patterns (off-hours usage)

Compare weekday vs weekend patterns

Score efficiency for each resource type

Return analysis report

End

4. compareBenchmarks()

Start

Input: farmID

Retrieve farm resource usage

Query industry benchmarks for region and farm size

Calculate percentage difference for each metric

Identify resources above/below average

Rank farm in percentile

Return comparison report

End

5. generateRecommendations()

Start

Input: farmID

Retrieve analysis and benchmark reports

Identify top 5 improvement opportunities

FOR each opportunity DO

Estimate implementation difficulty

Calculate expected savings

Assign priority score

END FOR

Sort by priority score

Format recommendations with action steps

Return recommendations list

End

6. calculateSavings()

Start

Input: recommendationID

Retrieve recommendation details

Estimate resource reduction percentage

Calculate cost savings = reduction * unit_price

Calculate environmental savings (carbon, water)

Estimate implementation cost

Calculate ROI = savings / implementation_cost

Return sav

3.2 Description for Component Payment Refund Processing

3.2.1 Processing narrative (PSPEC) for component Payment Refund Processing

The Payment Refund Processing component handles all refund requests for cancelled orders, returned products, or service failures. It validates refund eligibility, calculates refund amounts including partial refunds, coordinates with payment gateways, processes transactions, and updates financial records while maintaining compliance with payment regulations.

3.2.2 Component Payment Refund Processing interface description

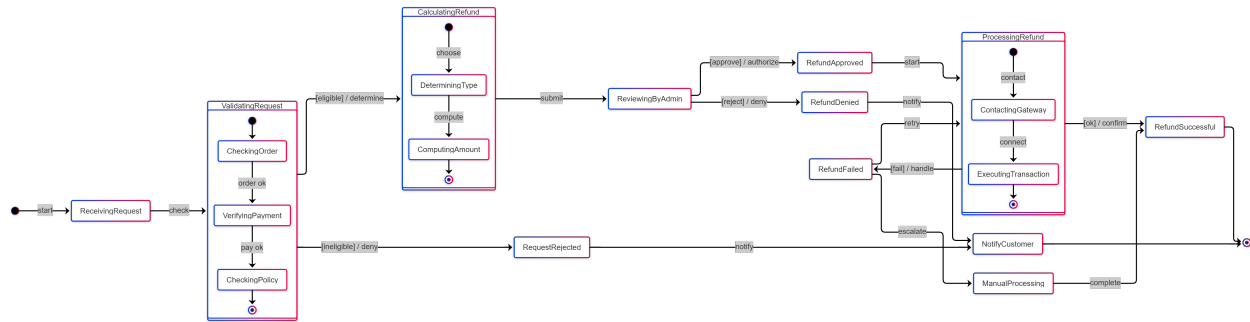
The interface allows customers to request refunds by selecting orders and specifying reasons. Administrators review refund requests with order details, payment information, and eligibility criteria displayed. The system shows refund status, processing timeline, and transaction history for transparency.

3.2.3 Component Payment Refund Processing processing detail

- Start
- Customer initiates refund request
- System validates eligibility and policies
- Refund amount calculated based on order status
- Administrator reviews and approves/rejects
- Payment gateway processes refund transaction

- Customer notified of refund status and timeline
- Financial records updated
- End

3.2.3.1 Design Class hierarchy Refund Processing



3.2.3.2 Restrictions/limitations for component Payment Refund Processing

Refunds only processed for orders placed within 90 days. Partial refunds require administrator approval. Cannot process refunds for orders already delivered more than 7 days ago. Maximum 3 refund requests per customer per month.

3.2.3.3 Performance issues for component Payment Refund Processing

High refund request volume may delay processing times. Payment gateway timeouts can cause transaction failures. Multiple retry attempts impact system responsiveness.

3.2.3.4 Design constraints for component Payment Refund Processing

Refunds take 3-5 business days to reflect in customer accounts. Processing fees non-refundable per policy. Refund amounts must match original payment method. Administrator review mandatory for refunds over \$200.

3.2.3.5 Processing detail for each operation of component Payment Refund Processing

1. `initiateRefund(orderID, reason)`: Starts refund request process
2. `validateEligibility(orderID)`: Checks refund policy compliance
3. `calculateRefundAmount(orderID, refundType)`: Computes refund value

4. `submitForApproval(refundRequest)`: Sends to administrator review
5. `approveRefund(refundID)`: Authorizes refund processing
6. `processTransaction(refundID)`: Executes refund through gateway
7. `retryFailedRefund(refundID)`: Attempts failed transaction again
8. `updateFinancialRecords(refundID)`: Updates accounting system

3.2.3.5.1 Processing narrative (PSPEC) for each operation

1. **`initiateRefund()`** - Creates refund request with customer details and reason
2. **`validateEligibility()`** - Verifies order status, delivery date, and refund policy compliance
3. **`calculateRefundAmount()`** - Determines full or partial refund based on order status and policy
4. **`submitForApproval()`** - Routes refund request to administrator for review
5. **`approveRefund()`** - Administrator authorizes refund after reviewing documentation
6. **`processTransaction()`** - Contacts payment gateway to reverse original transaction
7. **`retryFailedRefund()`** - Automatically retries failed transactions up to 3 times
8. **`updateFinancialRecords()`** - Records refund in accounting system and updates reports

3.2.3.5.2 Algorithmic model (PDL) for each operation

1. **`initiateRefund()`**

Start

Input: `orderId`, `reason`

Generate `refundID`

Retrieve order details

Set `refundDate` = current date

Set `status` = "Pending Validation"

Create `RefundRequest` record

Save to database

Return `refundID`

End

2. **`validateEligibility()`**

InnovaCode

Start

Input: orderID

Retrieve order and payment details

Calculate days since order date

IF days > 90 THEN Return "Outside refund window"

Check order status

IF status = "Delivered" THEN

 Calculate days since delivery

 IF days > 7 THEN Return "Delivery window expired"

END IF

Check customer refund history

IF refunds_this_month >= 3 THEN Return "Monthly limit reached"

Return "Eligible"

End

3. calculateRefundAmount()

Start

Input: orderID, refundType

Retrieve order total amount

IF refundType = "Full" THEN

 refundAmount = total amount

ELSE IF refundType = "Partial" THEN

 Get order status

 IF status = "Preparing" THEN

 refundAmount = total * 0.90 (10% restocking fee)

 ELSE IF status = "Shipped" THEN

 refundAmount = total * 0.80 (20% restocking fee)

END IF

InnovaCode

END IF

Subtract processing fee if applicable

Return refundAmount

End

4. submitForApproval()

Start

Input: refundRequest

IF refundAmount > 200 THEN

Find available administrator

Assign refund to admin

Set status = "Pending Approval"

Notify admin

ELSE

Call approveRefund(refundID) automatically

END IF

Return success

End

5. approveRefund()

Start

Input: refundID

Retrieve refund details

Validate admin authorization

Set status = "Approved"

Set approvalDate = current date

Update database

Call processTransaction(refundID)

InnovaCode

Return success

End

6. processTransaction()

Start

Input: refundID

Retrieve refund and payment details

Connect to payment gateway

Send refund request with original transaction ID

Wait for gateway response

IF response = "Success" THEN

Set status = "Completed"

Set completionDate = current date

Call updateFinancialRecords(refundID)

Notify customer "Refund processed"

Return success

ELSE

Set status = "Failed"

Return error

END IF

End

7. retryFailedRefund()

Start

Input: refundID

Retrieve refund details

Get retry count

IF retry_count >= 3 THEN

Set status = "Manual Processing Required"

Notify support team

Return "Escalated"

END IF

Increment retry_count

Wait 5 minute

Call processTransaction(refundID)

Return result

End

3.2 Description for Component AI Recipe Recommendation

3.2.1 Processing narrative (PSPEC) for component AI Recipe Recommendation

The AI Recipe Recommendation component analyzes purchased products and suggests recipes, meal plans, and cooking methods to customers. It considers seasonal availability, nutritional balance, dietary preferences, cooking skill level, and kitchen equipment to provide personalized culinary suggestions that help customers utilize their farm produce effectively.

3.2.2 Component AI Recipe Recommendation interface description

The interface displays recipe cards with images, ingredients, instructions, preparation time, and difficulty level. Customers filter recipes by dietary restrictions, cuisine type, or cooking time. The system shows which ingredients are available in current cart or past purchases and suggests complementary products needed to complete recipes.

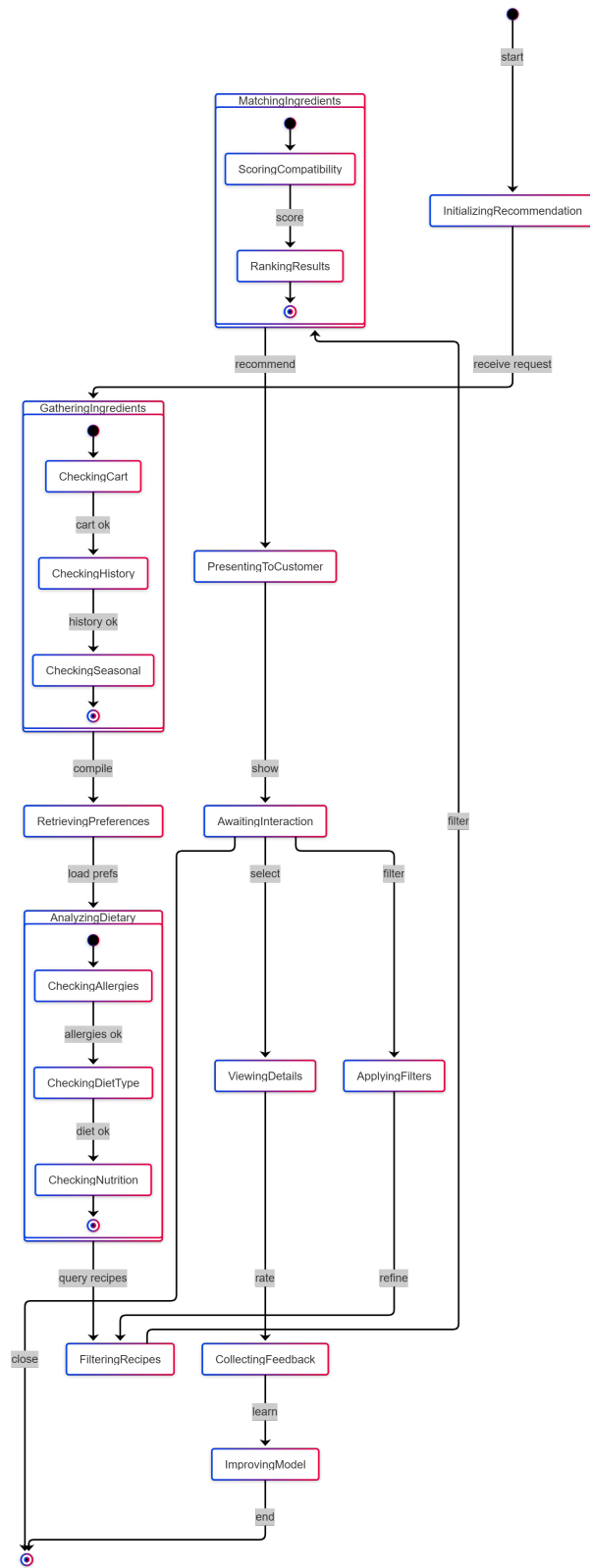
3.2.3 Component AI Recipe Recommendation processing detail

- Start
- Customer views purchased products or cart
- System retrieves product information and customer preferences
- AI analyzes ingredients and dietary requirements

InnovaCode

- Recipe database queried for matching recipes
- Recommendations ranked by relevance and seasonality
- Complementary products suggested
- Customer selects recipe and receives cooking instructions
- End

3.2.3.1 Design class hierarchy for AI Recipe Recommendation



3.2.3.2 Restrictions/limitations for component AI Recipe Recommendation

Recipe database limited to 10,000 verified recipes. Requires minimum 3 purchased products for accurate recommendations. Dietary preferences must be updated manually. Cannot guarantee allergen-free recipes without certification.

3.2.3.3 Performance issues for component AI Recipe Recommendation

Complex ingredient matching with large recipe databases causes delays. Real-time nutritional calculations resource-intensive. Multiple simultaneous recommendation requests slow response time.

3.2.3.4 Design constraints for component AI Recipe Recommendation

Recommendations limited to 20 recipes per request. Nutritional information accuracy $\pm 5\%$. Recipe images must be under 2MB. Complementary product suggestions capped at 5 items per recipe.

3.2.3.5 Processing detail for each operation of component AI Recipe Recommendation

1. `gatherIngredients(customerID)`: Collects available products from cart and purchase history
2. `retrievePreferences(customerID)`: Loads dietary restrictions and cooking preferences
3. `queryRecipes(ingredients, preferences)`: Searches recipe database with filters
4. `matchIngredients(recipeID, availableIngredients)`: Calculates ingredient compatibility
5. `rankRecommendations(recipes)`: Sorts by relevance, seasonality, and rating
6. `suggestComplements(recipeID)`: Identifies missing ingredients for purchase
7. `trackEngagement(customerID, recipeID)`: Monitors recipe views and usage
8. `collectFeedback(customerID, recipeID, rating)`: Gathers customer ratings

3.2.3.5.1 Processing narrative (PSPEC) for each operation

1. **`gatherIngredients()`** - Retrieves products from cart, recent purchases, and seasonal availability
2. **`retrievePreferences()`** - Loads allergies, diet type (vegan, vegetarian, etc.), and cooking skill level
3. **`queryRecipes()`** - Searches database for recipes matching available ingredients and preferences

4. **matchIngredients()** - Calculates percentage match between recipe requirements and available items
5. **rankRecommendations()** - Scores and sorts recipes by relevance, season, ratings, and preparation time
6. **suggestComplements()** - Identifies missing recipe ingredients available for purchase
7. **trackEngagement()** - Records which recipes customers view, save, and cook
8. **collectFeedback()** - Gathers ratings and reviews to improve future recommendations

3.2.3.5.2 Algorithmic model (PDL) for each operation

1. **gatherIngredients()**

Start

Input: customerID

Query cart for current items

Query purchase history (last 30 days)

Query seasonal products available

Combine all ingredients into list

Remove duplicates

Return ingredientList

End

2. **retrievePreferences()**

Start

Input: customerID

Query customer profile for allergies

Query dietary restrictions (vegan, gluten-free, etc.)

Query cooking skill level

Query cuisine preferences

Query equipment availability

Return preferences object

InnovaCode

End

3. queryRecipes()

Start

Input: ingredients, preferences

Search recipe database

Filter by dietary restrictions

Filter by allergen exclusions

Filter by skill level

Filter by available equipment

Return filtered recipe list

End

4. matchIngredients()

Start

Input: recipeID, availableIngredients

Retrieve recipe ingredients

Initialize matchCount = 0

FOR each recipe ingredient DO

IF ingredient in availableIngredients THEN

matchCount = matchCount + 1

END IF

END FOR

Calculate matchPercentage = (matchCount / totalIngredients) * 100

Return matchPercentage

End

5. rankRecommendations()

InnovaCode

Start

Input: recipes

FOR each recipe DO

 Calculate relevanceScore:

 - ingredientMatch * 0.4

 - seasonalBonus * 0.2

 - customerRating * 0.2

 - popularityScore * 0.1

 - freshnessBonus * 0.1

 Assign score to recipe

END FOR

Sort recipes by score descending

Return top 20 recipes

End

6. suggestComplements()

Start

Input: recipeID

Retrieve recipe ingredients

Retrieve customer's available ingredients

Initialize missingItems = []

FOR each recipe ingredient DO

 IF ingredient NOT in available THEN

 Add to missingItems

 END IF

END FOR

Query product database for missing items

Sort by price and availability

InnovaCode

Return top 5 complementary products

End

7. trackEngagement()

Start

Input: customerID, recipeID

Record view timestamp

Update recipe view count

IF customer saves recipe THEN

 Add to favorites

 Update preference model

END IF

IF customer adds missing items to cart THEN

 Record conversion

 Increase recipe relevance score

END IF

Return success

End

3.2 Description for Component Sustainability Dashboard

3.2.1 Processing narrative (PSPEC) for component Sustainability Dashboard

The Sustainability Dashboard component aggregates and visualizes environmental impact metrics across all farms including water usage, carbon footprint, waste reduction, energy consumption, and organic practices. It generates sustainability reports, compares performance against industry benchmarks, and tracks progress toward environmental goals.

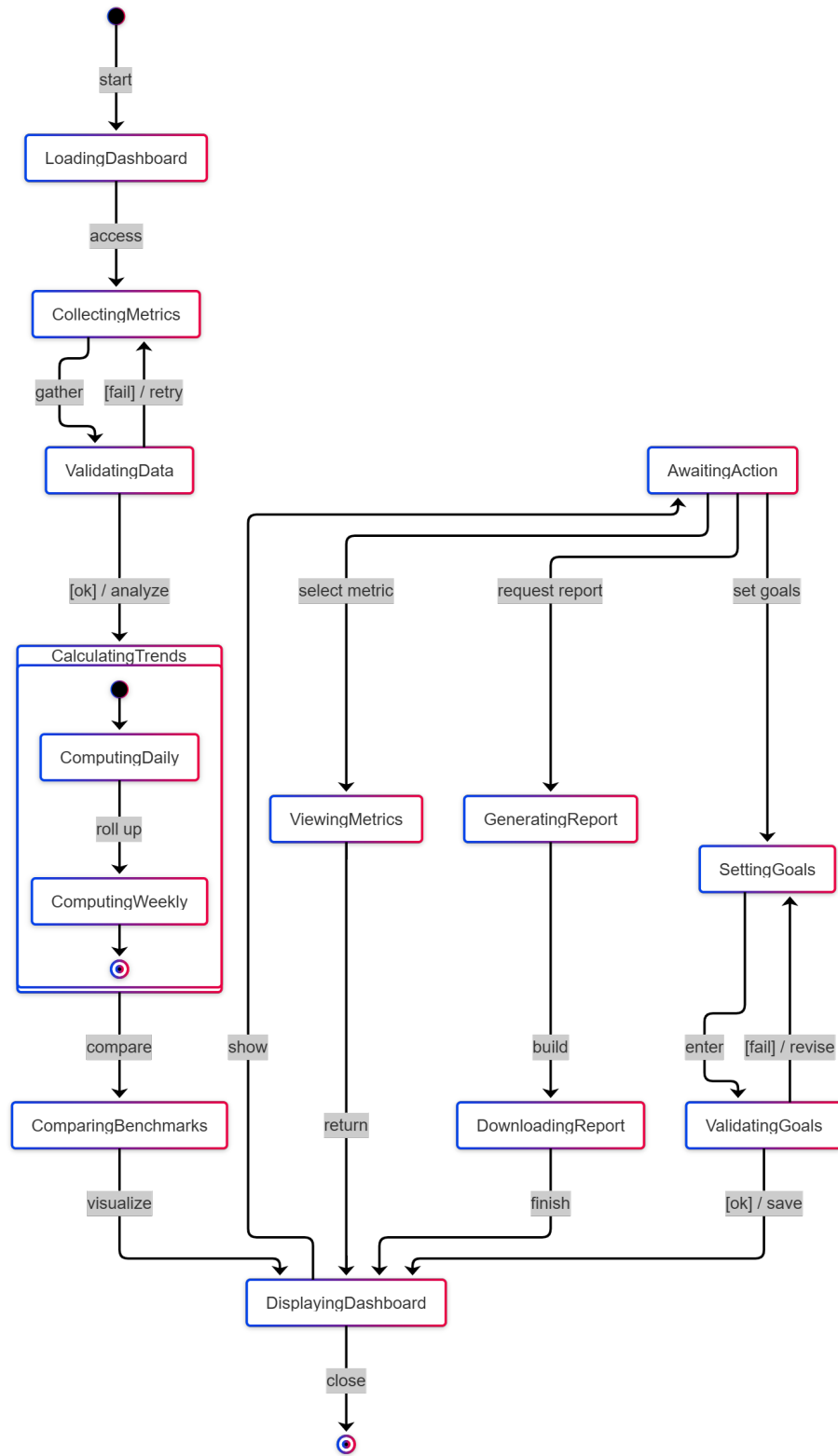
3.2.2 Component Sustainability Dashboard interface description

The interface displays interactive charts, graphs, and metrics showing sustainability performance over time. Farmers and investors view water conservation percentages, carbon savings, waste diversion rates, and energy efficiency scores. The system provides downloadable reports, goal-setting tools, and achievement badges for reaching sustainability milestones.

3.2.3 Component Sustainability Dashboard processing detail

- Start
- System collects sustainability metrics from farms
- Data validated and normalized for comparison
- Historical trends calculated and analyzed
- Benchmarks retrieved from industry standards
- Visual representations generated with insights
- Reports compiled for stakeholders
- Achievements recognized and goals updated
- End

3.2.3.2 Design class hierarchy for Sustainability Dashboard



3.2.3.2 Restrictions/limitations for component Sustainability Dashboard

Metrics updated daily at midnight only. Historical data limited to 2 years. Benchmark comparisons available for registered regions only. Reports generated in PDF format exclusively. Maximum 5 custom goals per farm.

3.2.3.3 Performance issues for component Sustainability Dashboard

Large data aggregation across multiple farms causes loading delays. Complex chart rendering slows dashboard display. Report generation for annual data may take several minutes.

3.2.3.4 Design constraints for component Sustainability Dashboard

Dashboard displays maximum 12 metrics simultaneously. Chart data points limited to 365 days. Report file size capped at 10MB. Goal progress calculated weekly. Sustainability scores updated monthly only.

3.2.3.5 Processing detail for each operation of component Sustainability Dashboard

1. `collectMetrics(farmID)`: Retrieves water, carbon, waste, energy data from sensors
2. `validateData(metricsData)`: Checks data completeness and accuracy
3. `calculateTrends(farmID, timeRange)`: Computes daily, weekly, monthly trends
4. `compareBenchmarks(farmID)`: Compares metrics against industry standards
5. `generateVisuals(metricsData)`: Creates charts and graphs
6. `generateReport(farmID, reportType)`: Compiles comprehensive sustainability report
7. `setGoal(farmID, metricType, targetValue)`: Defines sustainability targets
8. `trackProgress(goalID)`: Monitors goal advancement

3.2.3.5.1 Processing narrative (PSPEC) for each operation

1. **`collectMetrics()`** - Retrieves water, carbon, waste, and energy data from farm sensors and manual inputs
2. **`validateData()`** - Checks data completeness, accuracy, and reasonable ranges
3. **`calculateTrends()`** - Computes daily, weekly, monthly trends and identifies improvement patterns
4. **`compareBenchmarks()`** - Compares farm metrics against industry standards and similar farms

5. **generateVisuals()** - Creates charts, graphs, and visual representations of sustainability data
6. **generateReport()** - Compiles comprehensive sustainability report with insights and recommendations
7. **setGoal()** - Defines sustainability targets with timelines and tracks achievement milestones
8. **trackProgress()** - Monitors goal advancement and calculates percentage completion

3.2.3.5.2 Algorithmic model (PDL) for each operation

1. collectMetrics()

Start

Input: farmID

Query database for water usage data

Query database for carbon emission data

Query database for waste generation data

Query database for energy consumption data

Aggregate all metrics into MetricsData object

Return metricsData

End

2. validateData()

Start

Input: metricsData

Check all required fields present

IF missing fields THEN Return false

Validate value ranges (water ≥ 0 , carbon ≥ 0 , etc.)

IF invalid ranges THEN Return false

Check timestamp consistency

IF inconsistent THEN Return false

Return true

InnovaCode

End

3. calculateTrends()

Start

Input: farmID, timeRange

Retrieve historical metrics for timeRange

Calculate daily averages

Compute weekly trends

Identify improvement or decline patterns

Calculate percentage changes

Store trends in TrendData object

Return trendData

End

4. compareBenchmarks()

Start

Input: farmID

Retrieve farm metrics

Query industry benchmarks for region

Calculate differences for each metric

Determine percentile ranking

Identify areas above/below standard

Create ComparisonResult object

Return comparisonResult

End

5. generateVisuals()

Start

InnovaCode

Input: metricsData

Create line chart for water usage over time

Create bar chart for carbon savings

Create pie chart for waste breakdown

Create gauge for energy efficiency score

Format charts with labels and legends

Store in ChartData object

Return chartData

End

6. generateReport()

Start

Input: farmID, reportType

Collect all metrics for farm

Calculate summary statistics

Generate trend analysis

Compare with benchmarks

Create visualizations

Compile text insights and recommendations

Format as PDF document

Return report

End

7. setGoal()

Start

Input: farmID, metricType, targetValue

Retrieve current metric value

Validate target is achievable (within 50% improvement)

InnovaCode

IF unrealistic THEN Return error

Generate goalID

Set startDate = current date

Set deadline based on target difficulty

Create Goal record in database

Return goal

End

8. trackProgress()

Start

Input: goalID

Retrieve goal details

Get current metric value

Calculate $\text{progress} = (\text{current} - \text{start}) / (\text{target} - \text{start}) * 100$

Determine status (On Track, Behind, Achieved)

Update progress in database

Return progressData

End

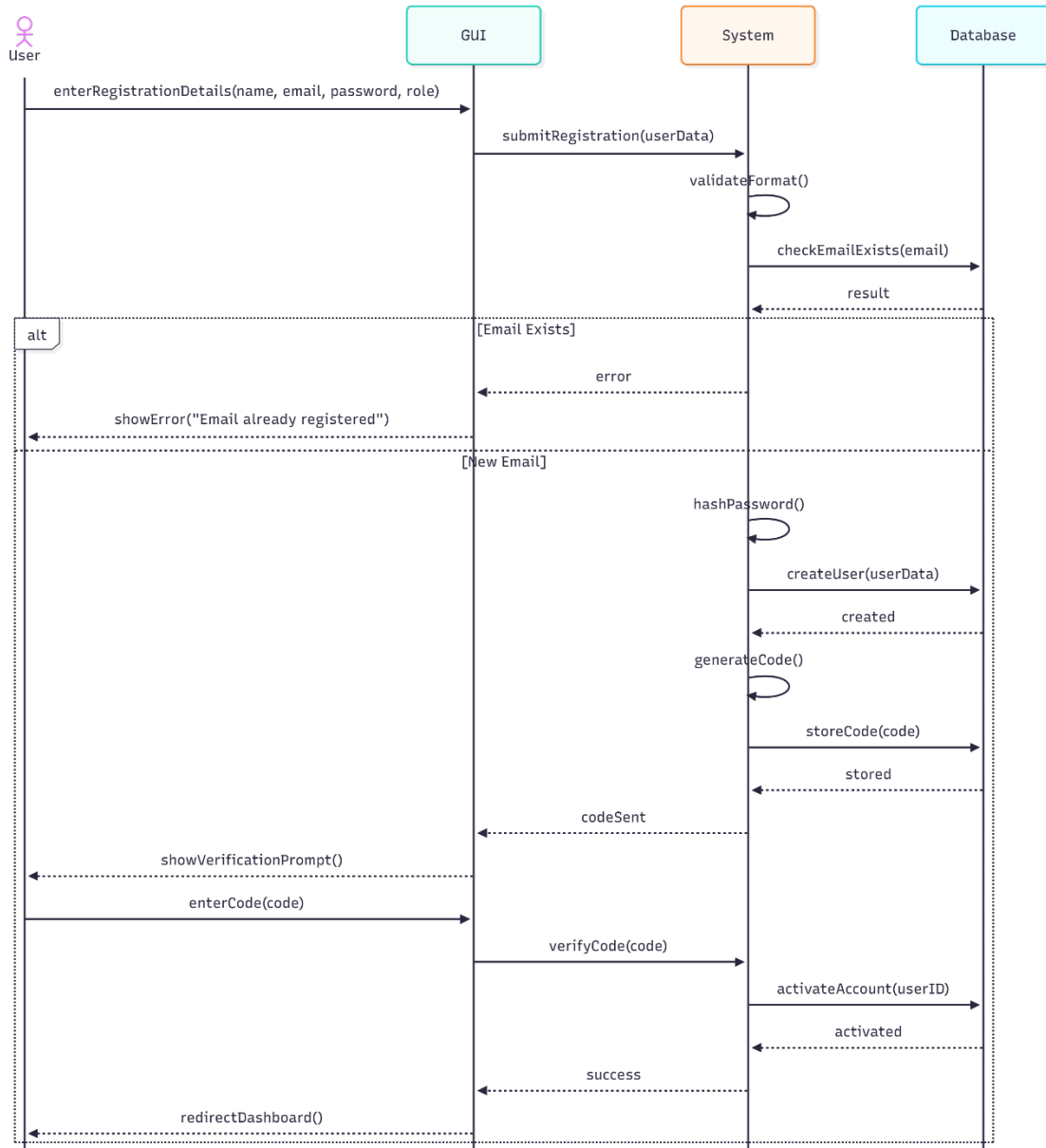
3.3 Dynamic Behavior for Components

3.3.1 Interaction Diagrams

Here are shorter descriptions for all 15 sequence diagrams:

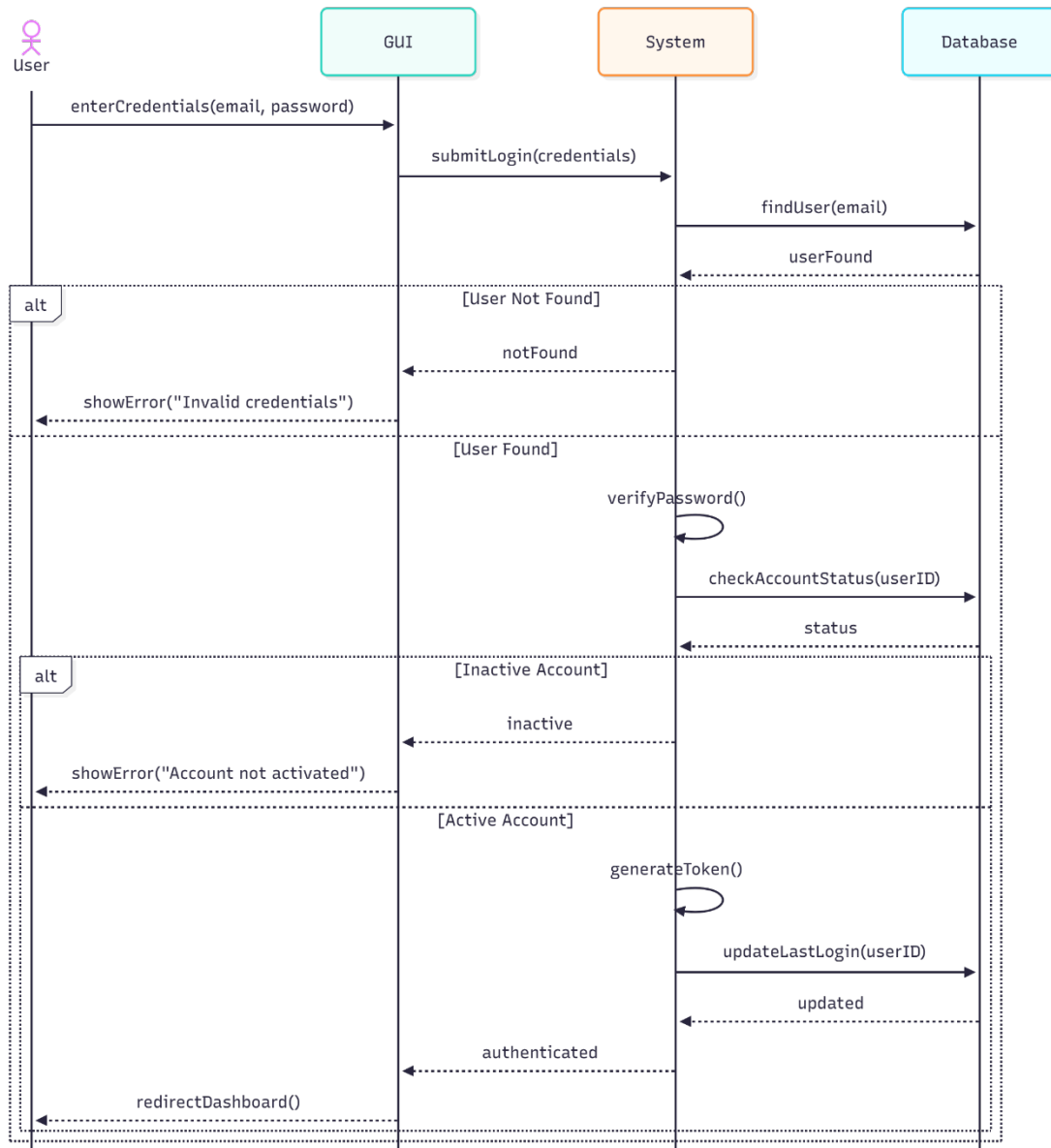
1) User Registration Process

The registration sequence describes how new users join the Shared Farm System. Users enter their details (name, email, password, role) through the GUI. The system validates the format and checks if the email exists in the database. If the email is already registered, an error is shown. For new emails, the system hashes the password, creates the user account, generates a verification code, and stores it in the database. The code is sent to the user's email. After the user enters the code, the system activates the account and redirects them to their role-specific dashboard.



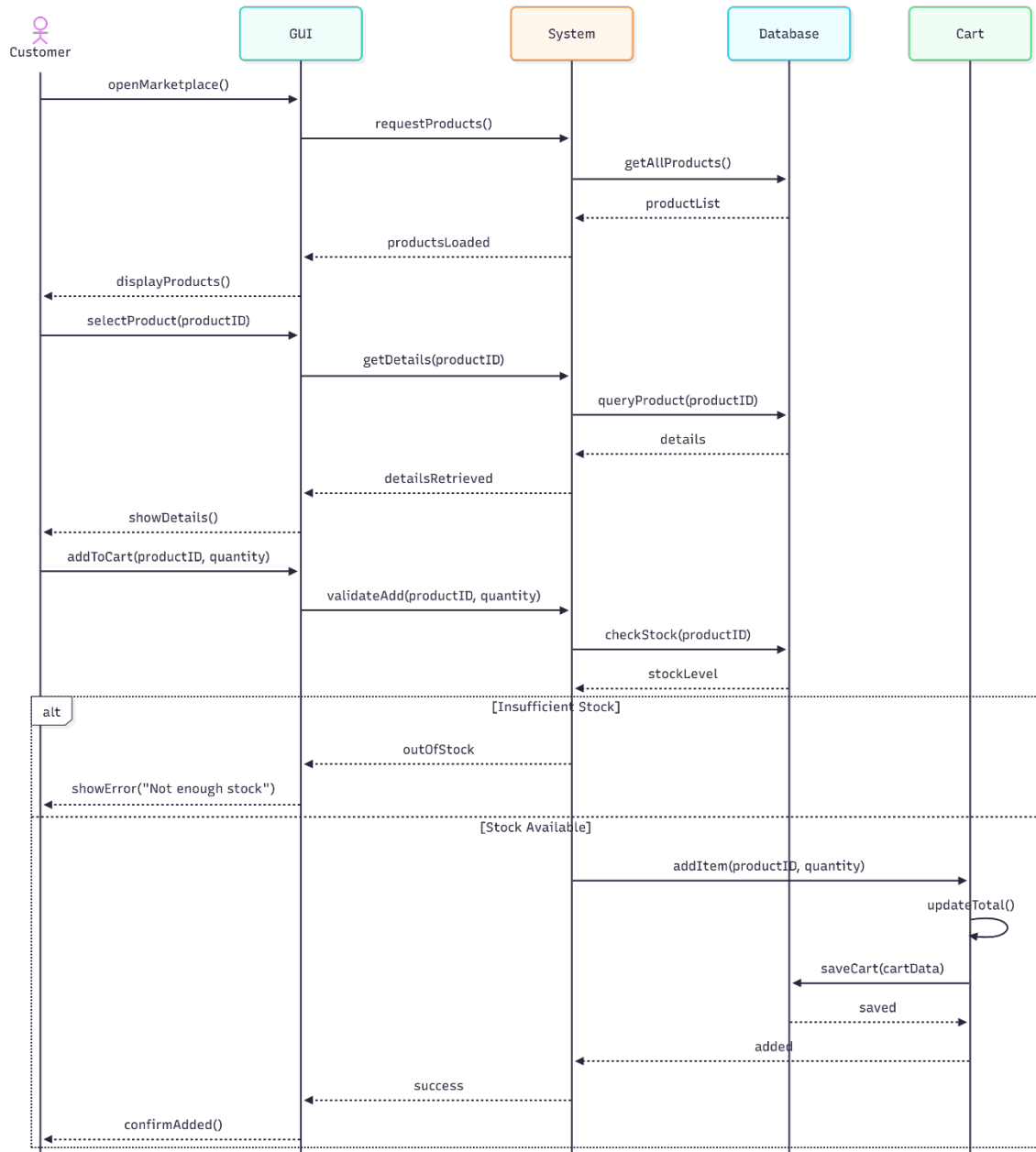
2) User Login Process

The login sequence shows how users access their accounts. Users enter email and password through the GUI. The system queries the database to find the user. If not found, an error is displayed. If found, the system verifies the password and checks account status in the database. Inactive accounts receive an error message. For active accounts, the system generates a session token, updates the last login timestamp in the database, and redirects the user to their dashboard.



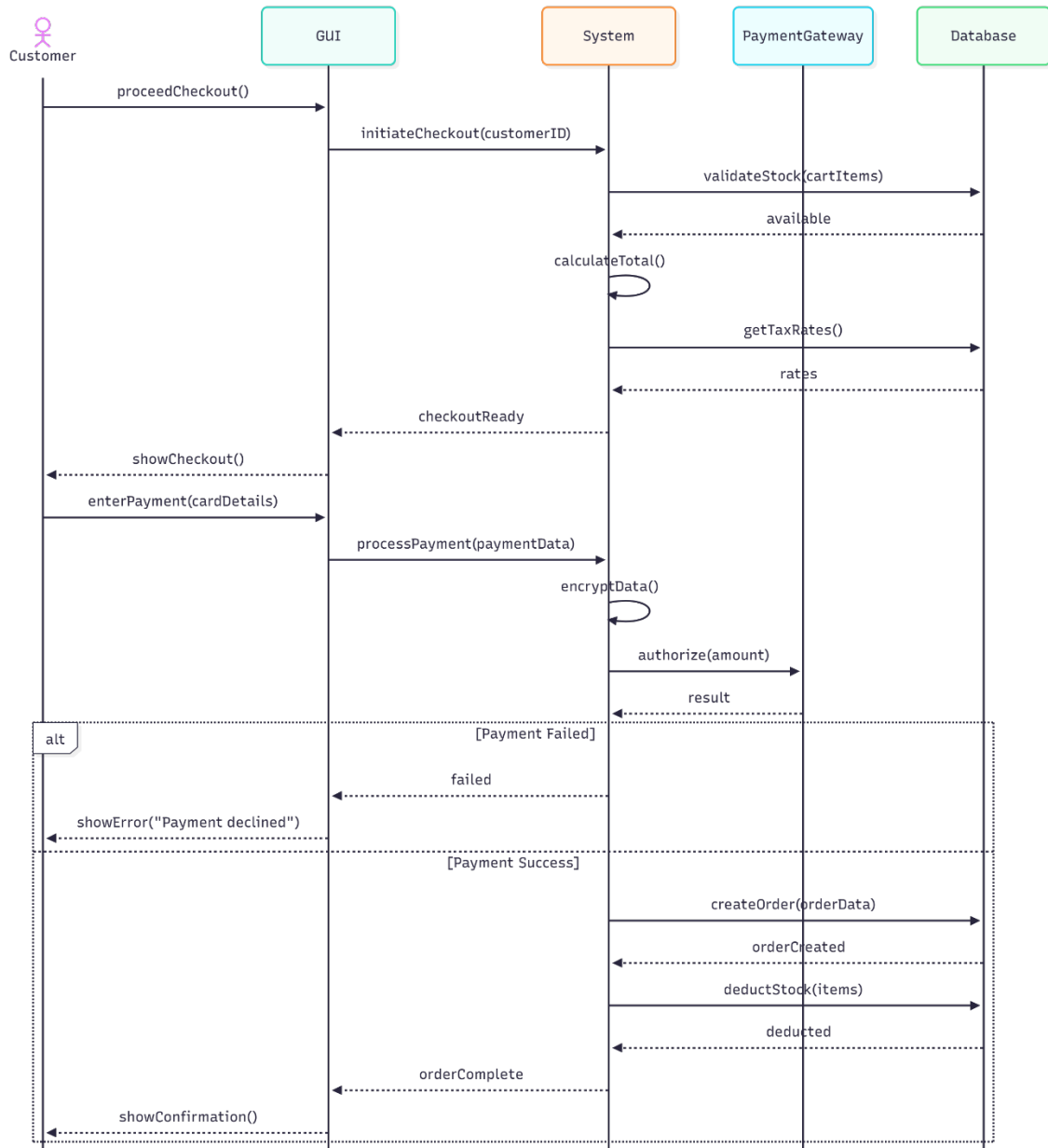
3) Browse Products and Add to Cart

This sequence shows how customers browse and purchase farm produce. The customer opens the marketplace, and the system retrieves all products from the database and displays them. When a customer selects a product, detailed information is fetched from the database. The customer adds items to cart with a specified quantity. The system checks stock availability in the database. If insufficient stock exists, an error is shown. If stock is available, the cart component adds the item, updates the total, saves cart data to the database, and confirms the addition to the customer.



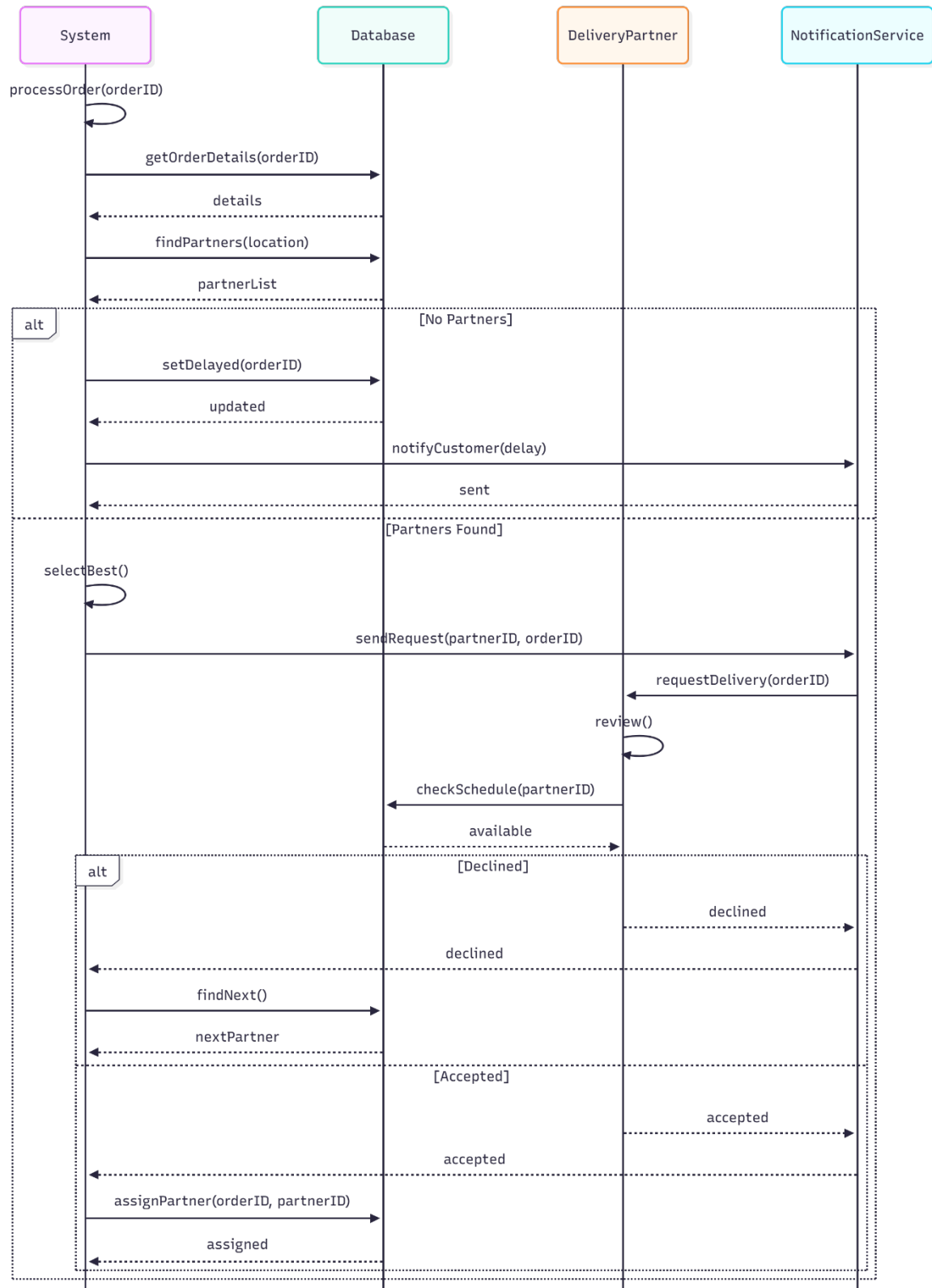
4) Complete Order and Payment

The checkout sequence illustrates the payment process. The customer proceeds to checkout, and the system validates stock in the database. It calculates the total including taxes (retrieved from database) and delivery fees. The customer enters payment details, which the system encrypts and sends to the Payment Gateway. If payment fails, an error is shown. On success, the system creates an order record in the database, deducts stock quantities, and displays order confirmation to the customer.



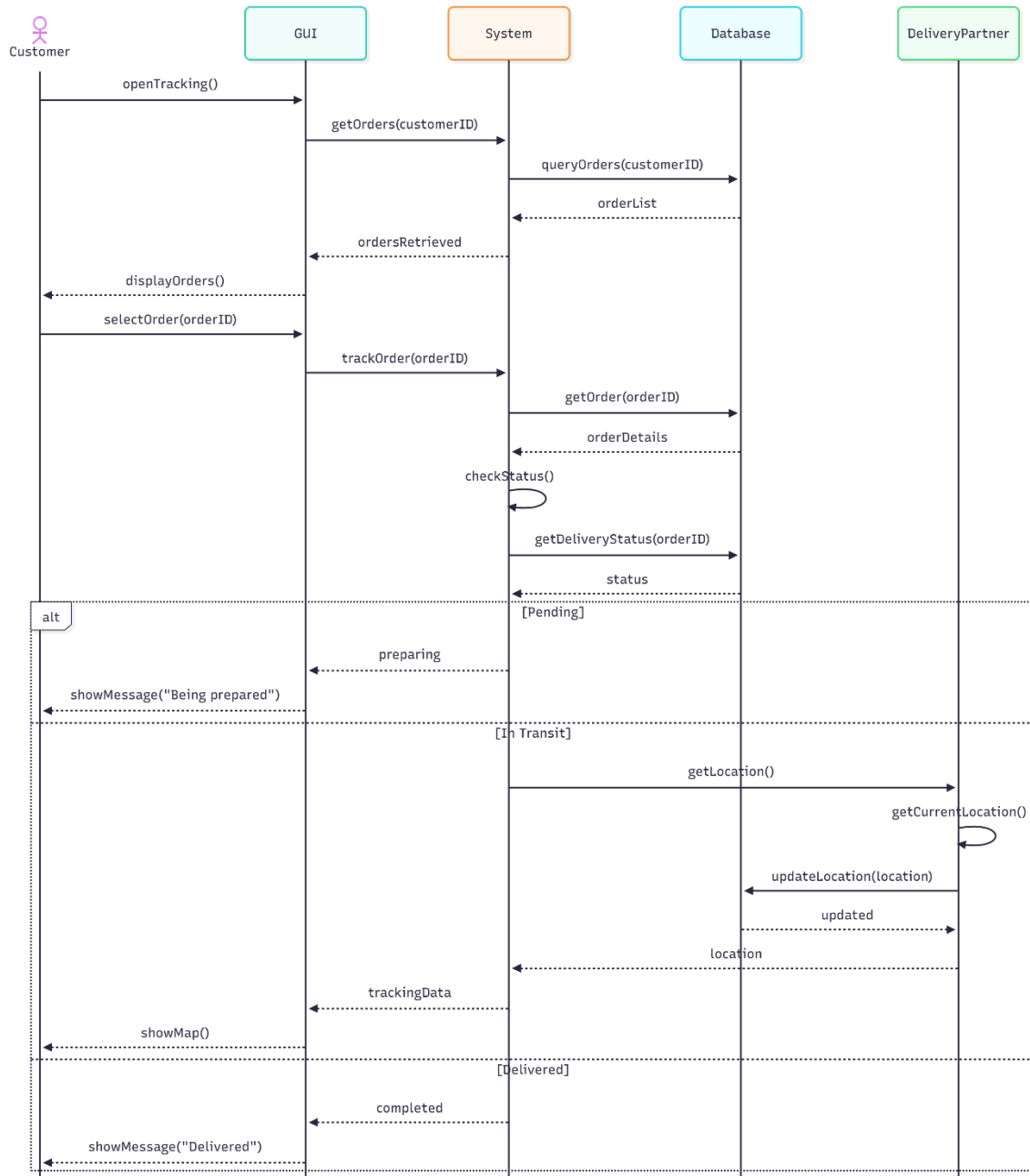
5) Assign Delivery Partner

This sequence shows automatic delivery partner assignment. The system processes the new order, retrieves order details from the database, and searches for available partners near the delivery location. If none are available, the order is marked "Delayed" and the customer is notified. If partners exist, the system selects the best one and sends a request. The partner reviews the request and checks their schedule in the database. If declined, the system finds another partner. If accepted, the order is updated in the database with the assigned partner ID.



6) Track Order Delivery

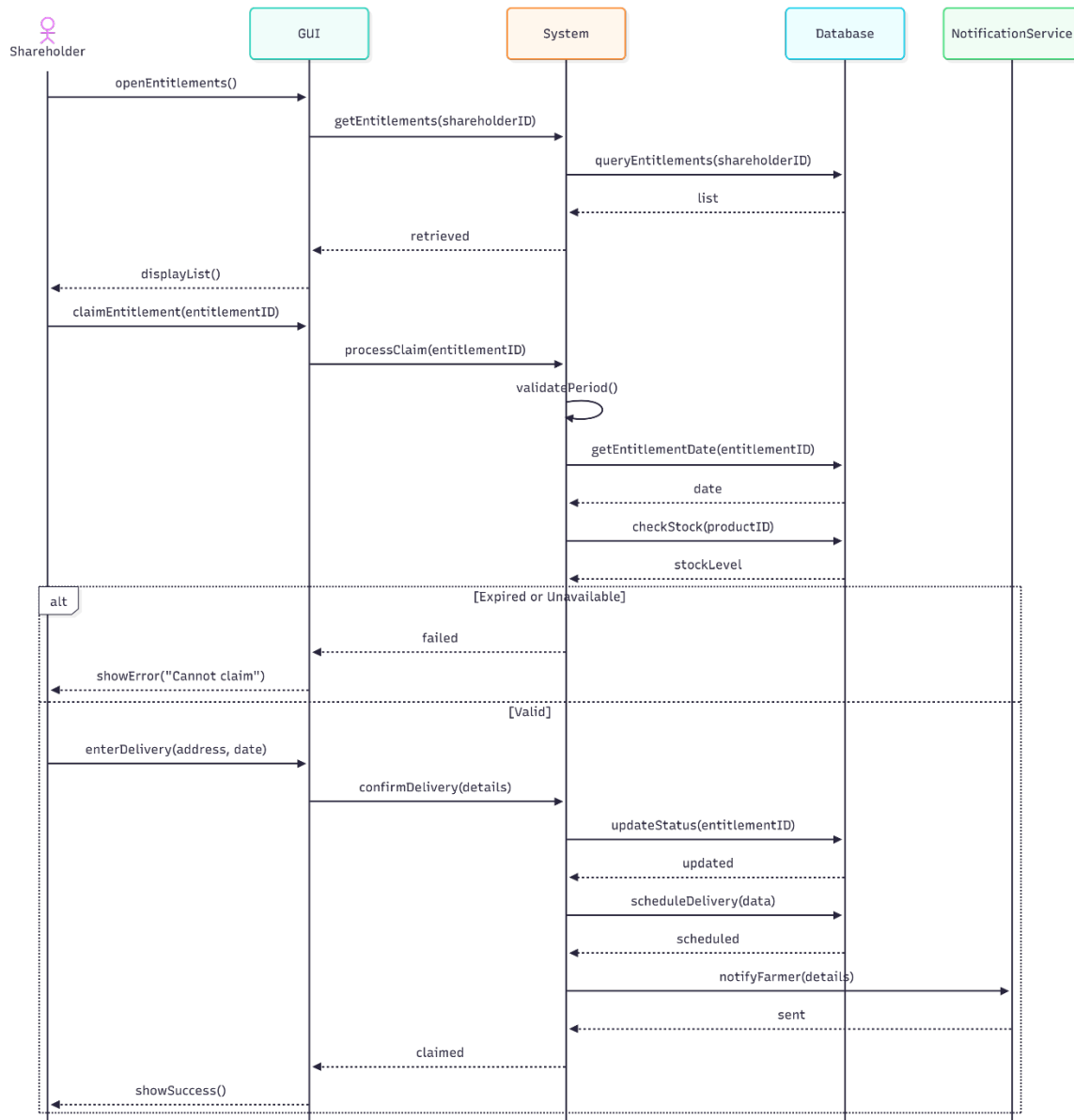
The tracking sequence shows how customers monitor deliveries. The customer opens tracking, and the system retrieves active orders from the database. After selecting an order, the system fetches order details and checks delivery status from the database. For pending orders, a "Being prepared" message is shown. For in-transit orders, the system requests location from the delivery partner, who retrieves their GPS position, updates it in the database, and sends it to the system for display on a map. Delivered orders show a completion message.



7) Shareholder Claims Entitlement

This sequence shows how shareholders claim their produce allocations. The shareholder opens entitlements, and the system retrieves available entitlements from the database. After selecting one to claim, the system validates the claim period by checking the issue date in the database and verifies stock availability. If expired or unavailable, an error is shown. For valid claims, the

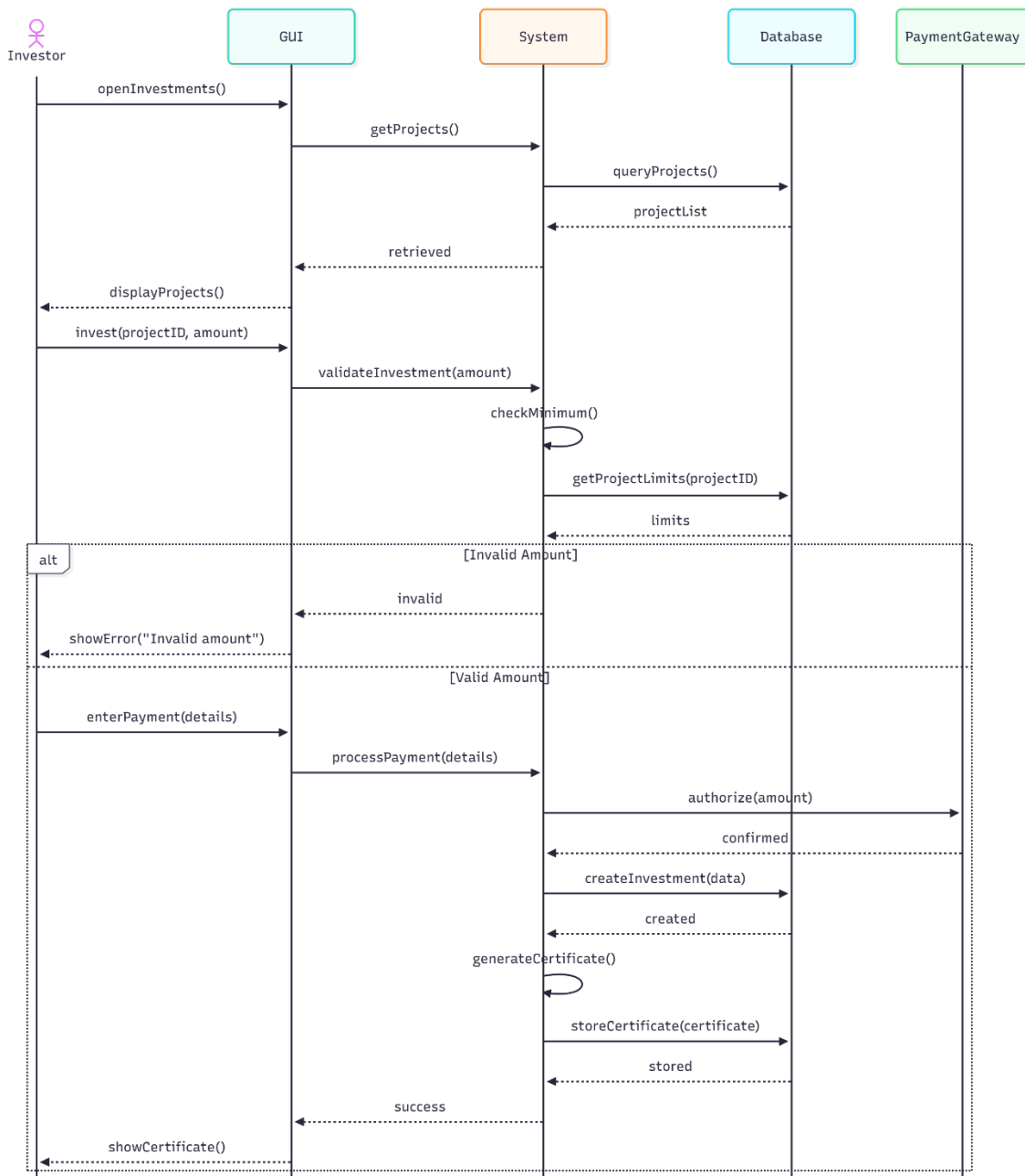
shareholder enters delivery details. The system updates entitlement status in the database, creates a delivery schedule, notifies the farmer, and confirms the claim.



8) Investor Funds Farm Project

The investment sequence shows how investors fund agricultural projects. The investor opens the portal, and the system retrieves active projects from the database. After selecting a project and entering an amount, the system validates it against minimum requirements retrieved from the database. Invalid amounts trigger an error. For valid investments, the investor enters payment

details. The Payment Gateway confirms the transaction. The system creates an investment record, generates and stores a certificate in the database, and displays it to the investor.

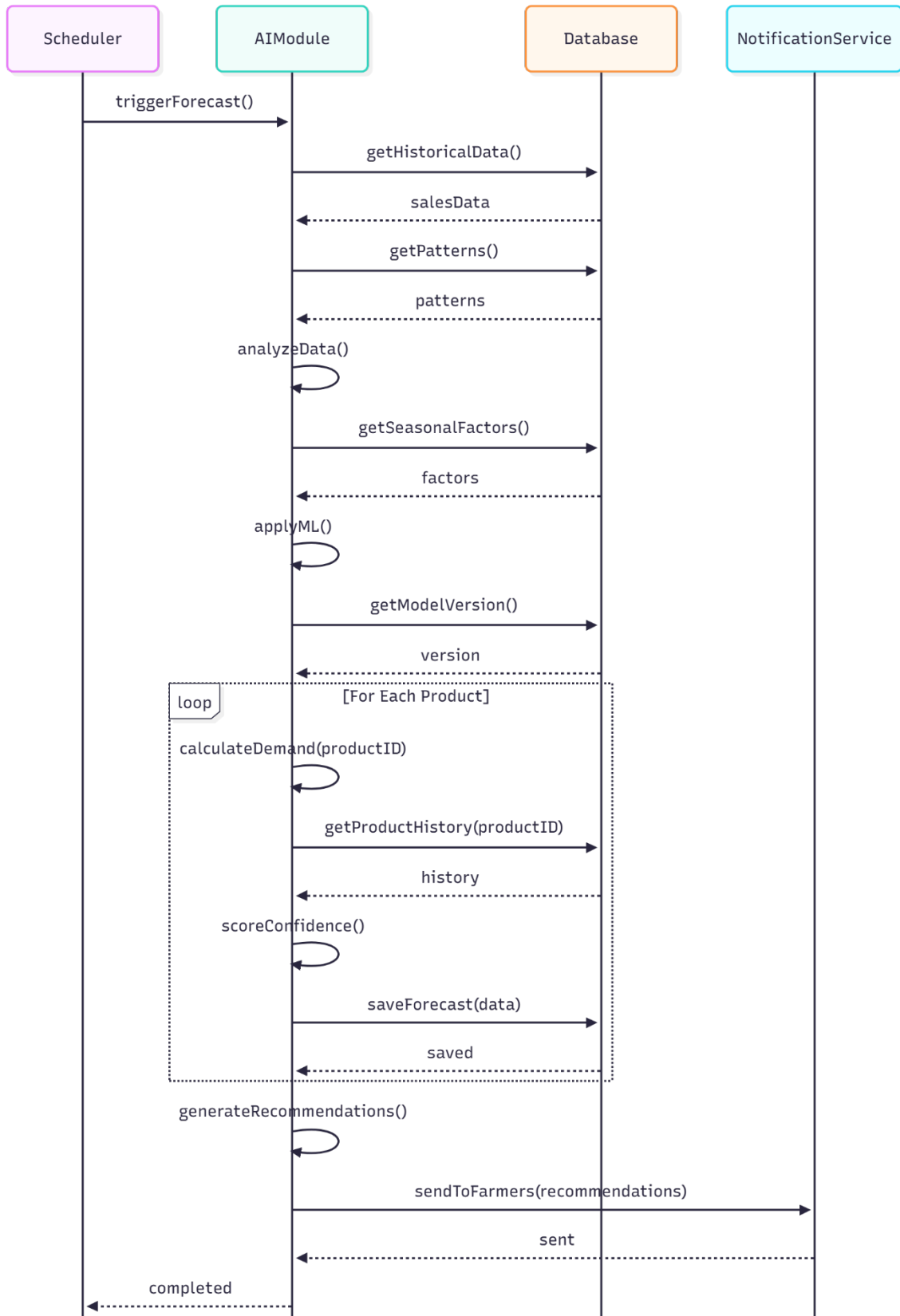


9) AI Generates Demand Forecast

This sequence demonstrates AI-driven demand prediction. The scheduler triggers the AI Module, which retrieves historical sales data and seasonal patterns from the database. The module analyzes data and retrieves seasonal factors and model versions from the database before applying machine

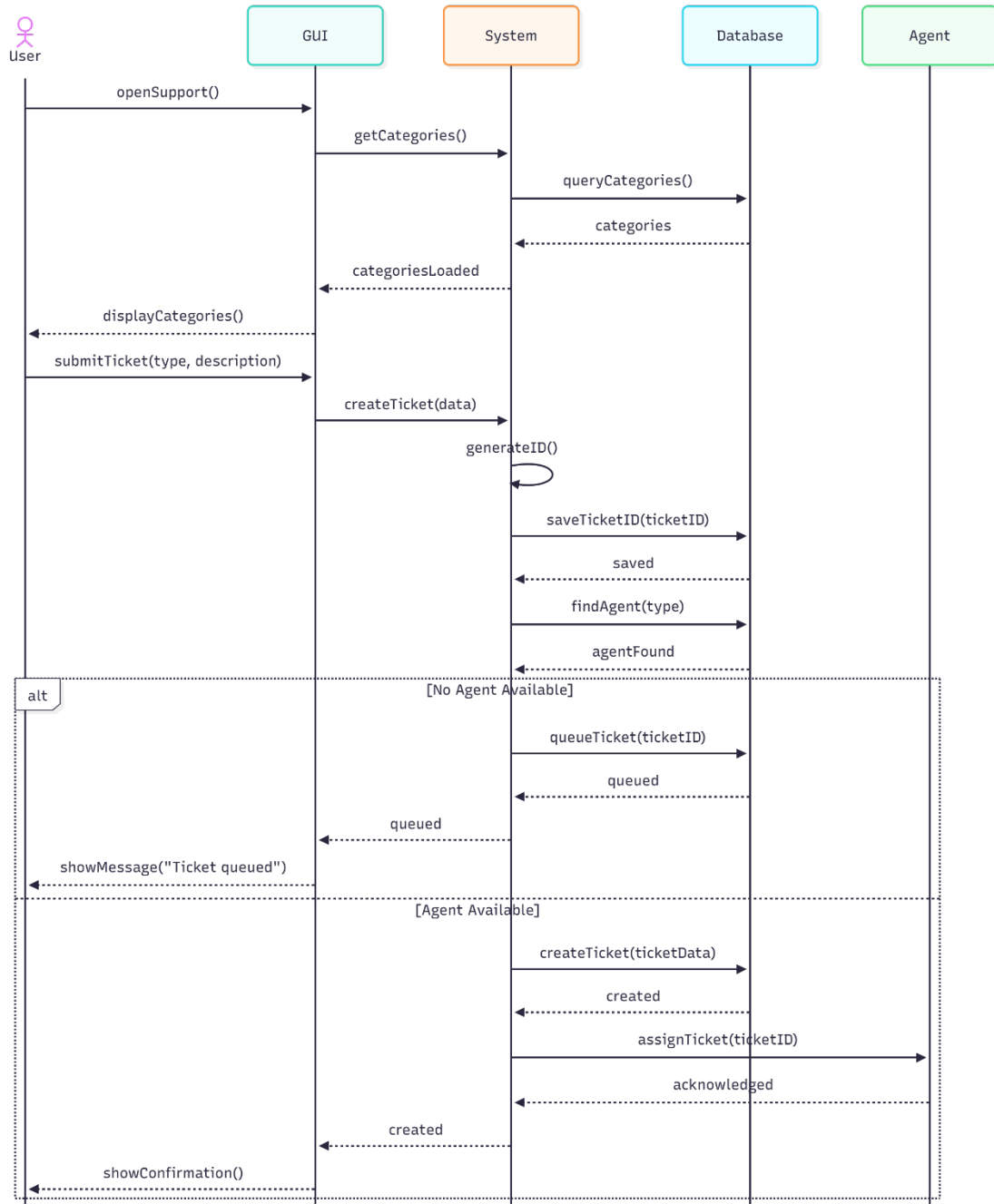
InnovaCode

learning algorithms. For each product, it calculates predicted demand by retrieving product history from the database, scores confidence, and saves the forecast. After processing all products, recommendations are generated and sent to farmers, and the module reports completion to the scheduler.



10) Submit Support Ticket

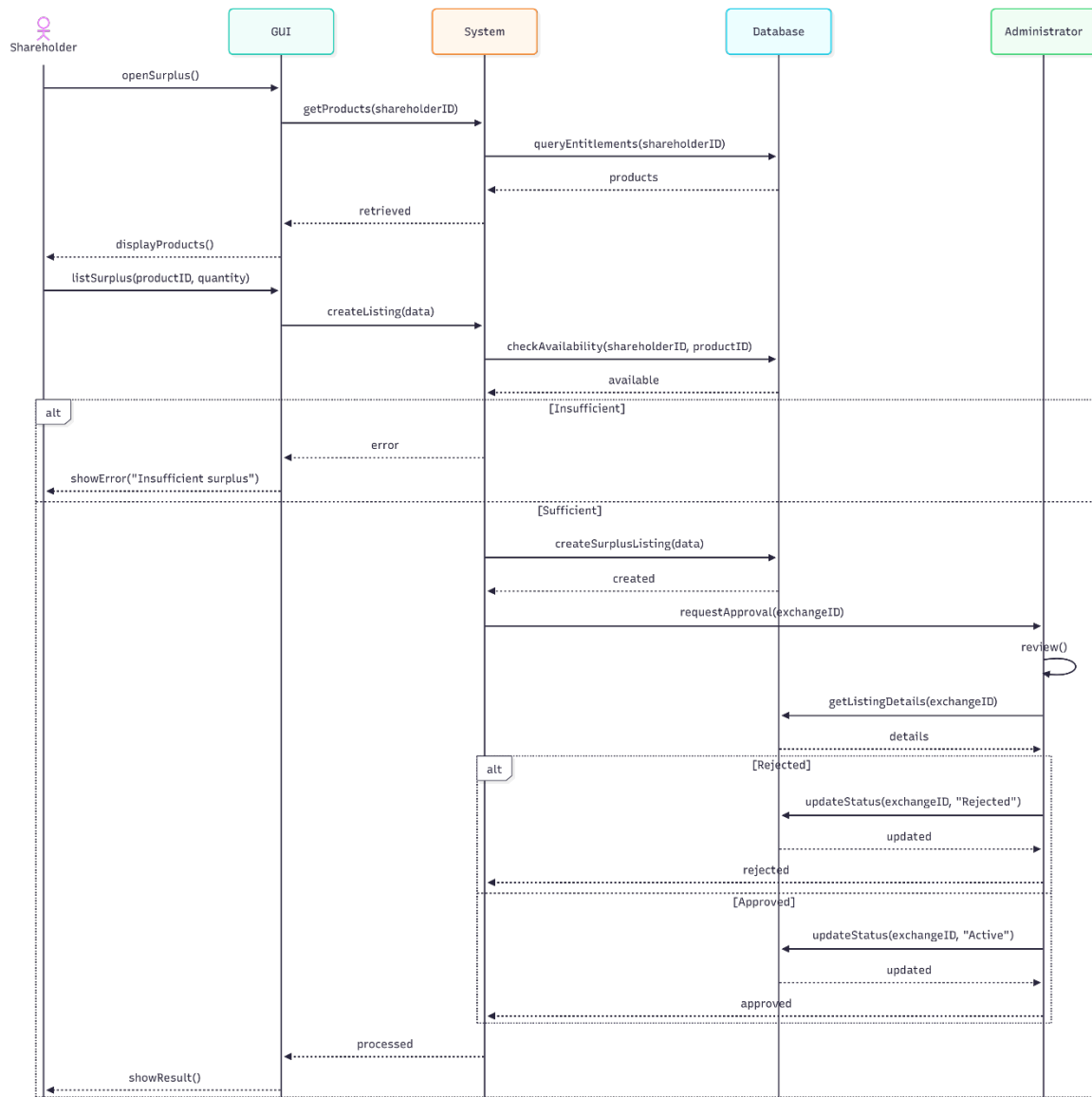
The support ticket sequence shows how users request help. Users open support, and the system retrieves issue categories from the database. After selecting a category and entering details, the system generates a ticket ID, saves it to the database, and searches for available agents. If none are available, the ticket is queued in the database with a status message to the user. If an agent is available, the ticket is created in the database, assigned to the agent who acknowledges it, and confirmation is shown to the user.



11) List Surplus Produce

This sequence shows shareholders listing excess produce. The shareholder opens surplus section, and the system retrieves their entitlements from the database. After selecting a product and quantity, the system checks availability in the database. If insufficient, an error is shown. For sufficient surplus, a listing is created in the database and sent to an administrator for approval. The

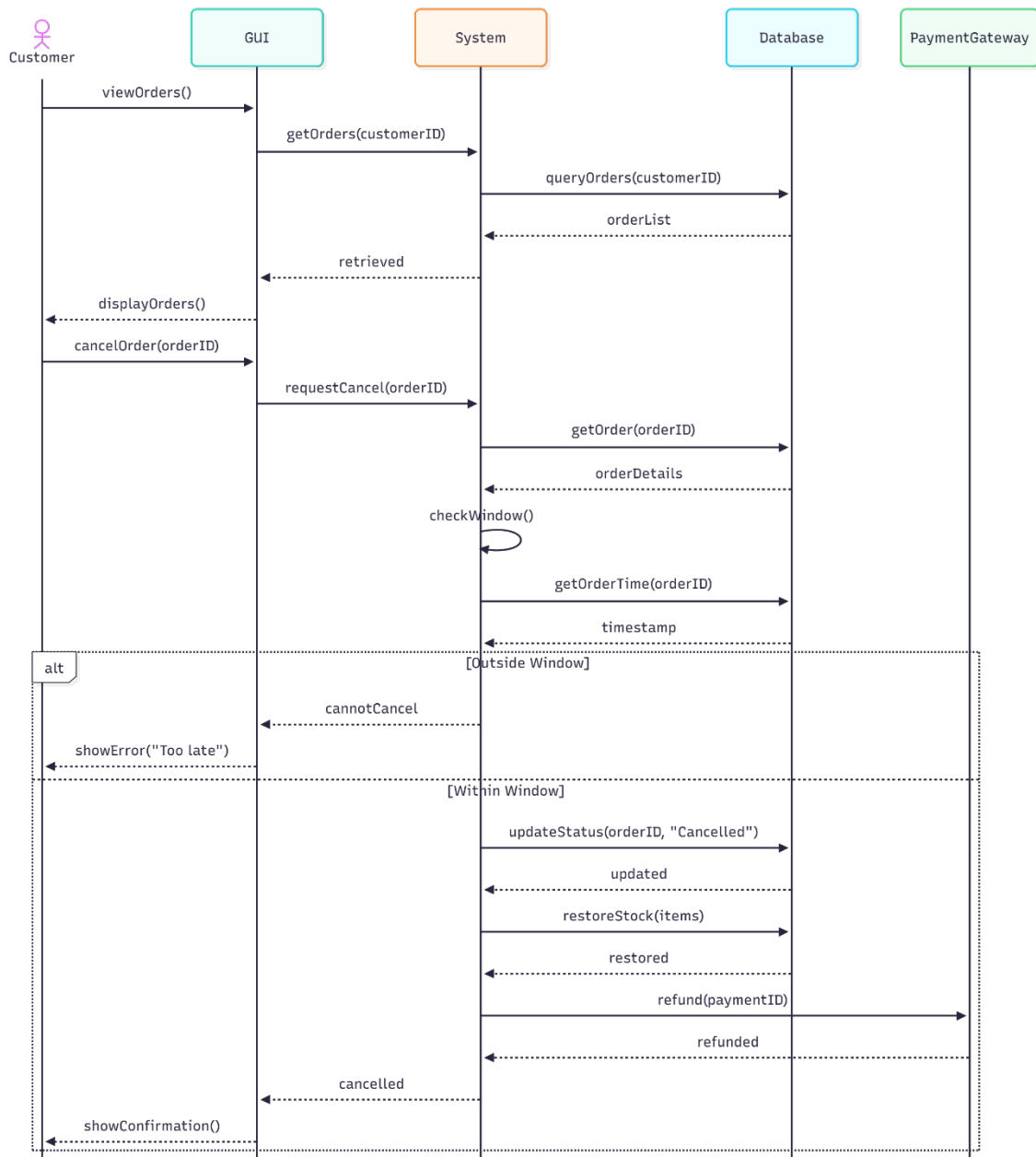
administrator reviews it, retrieves listing details from the database, and either rejects or approves by updating the status in the database. The result is displayed to the shareholder.



12) Cancel Order

The cancellation sequence shows how customers cancel orders. The customer views orders retrieved from the database. After selecting an order to cancel, the system fetches details and checks the cancellation window by querying the order timestamp from the database. If outside the window, an error is shown. Within the window, the system updates order status to "Cancelled" in

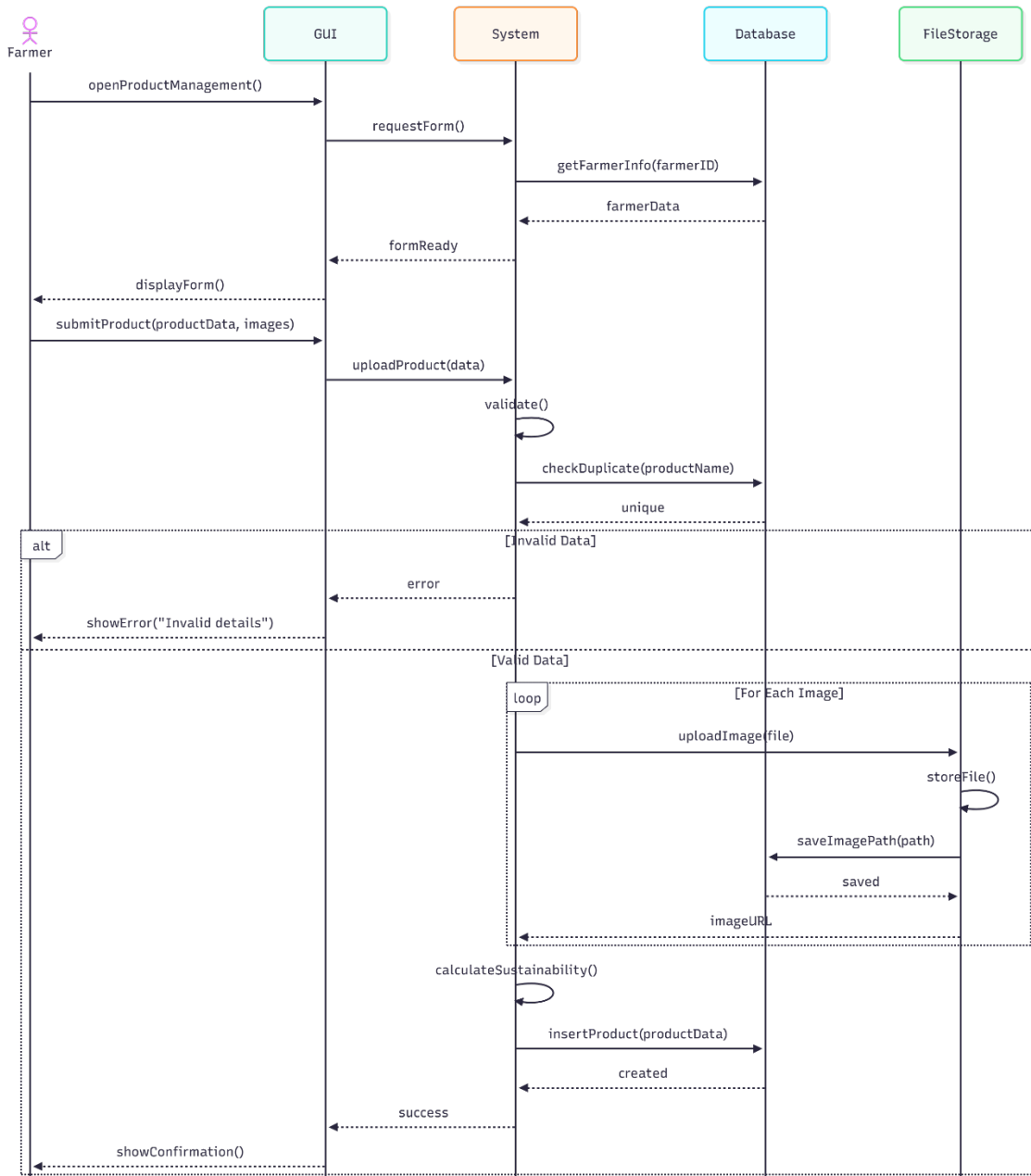
the database, restores stock quantities, processes a refund through the Payment Gateway, and displays confirmation to the customer.



13) Farmer Uploads Product

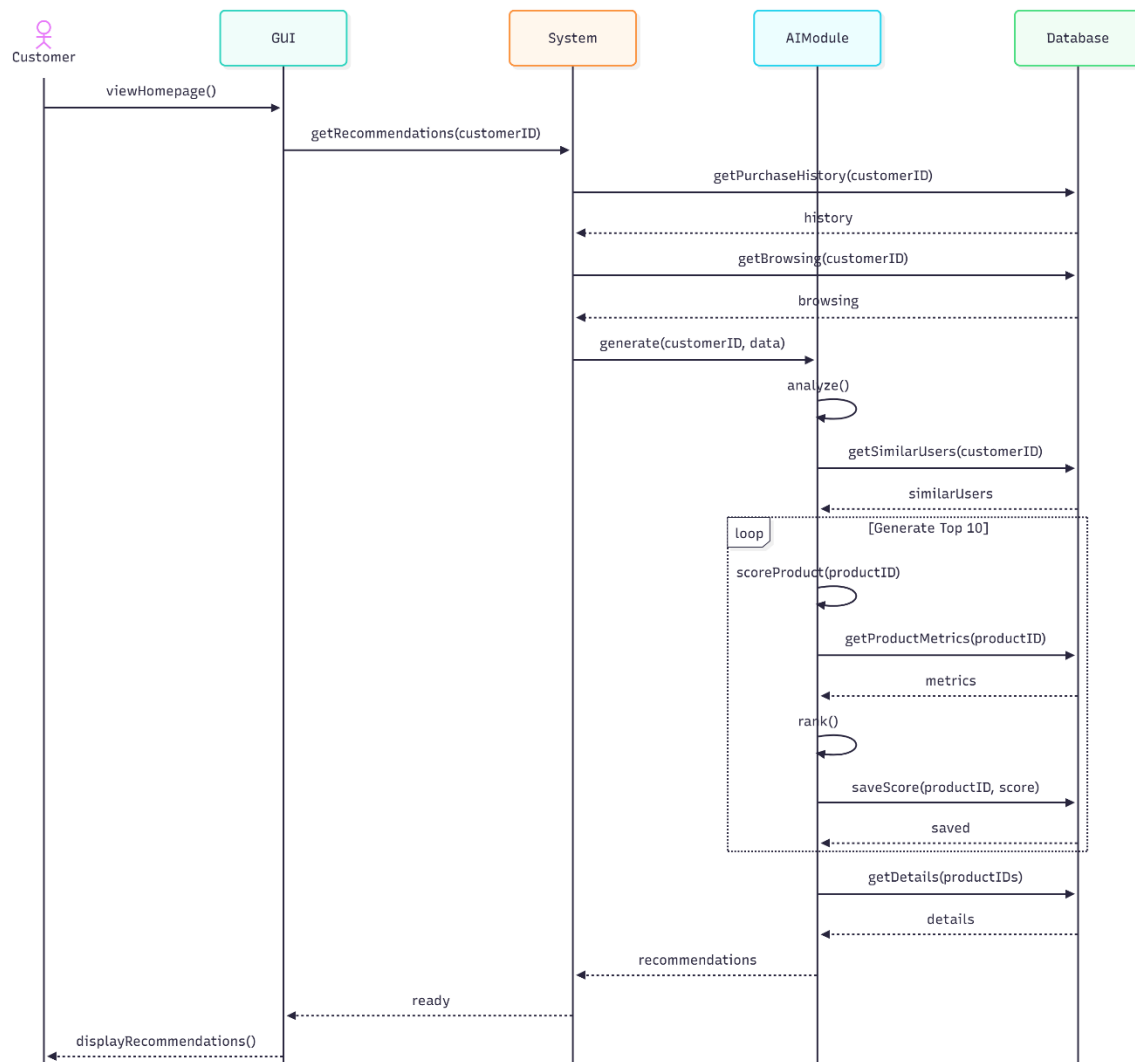
This sequence shows farmers adding products to the marketplace. The farmer opens product management, and the system retrieves farmer information from the database to display the upload form. After submitting product details and images, the system validates data and checks for

duplicates in the database. Invalid data triggers an error. For valid data, each image is uploaded to File Storage, which stores the file, saves the path in the database, and returns the URL. The system calculates sustainability score, inserts the product in the database, and confirms success to the farmer.



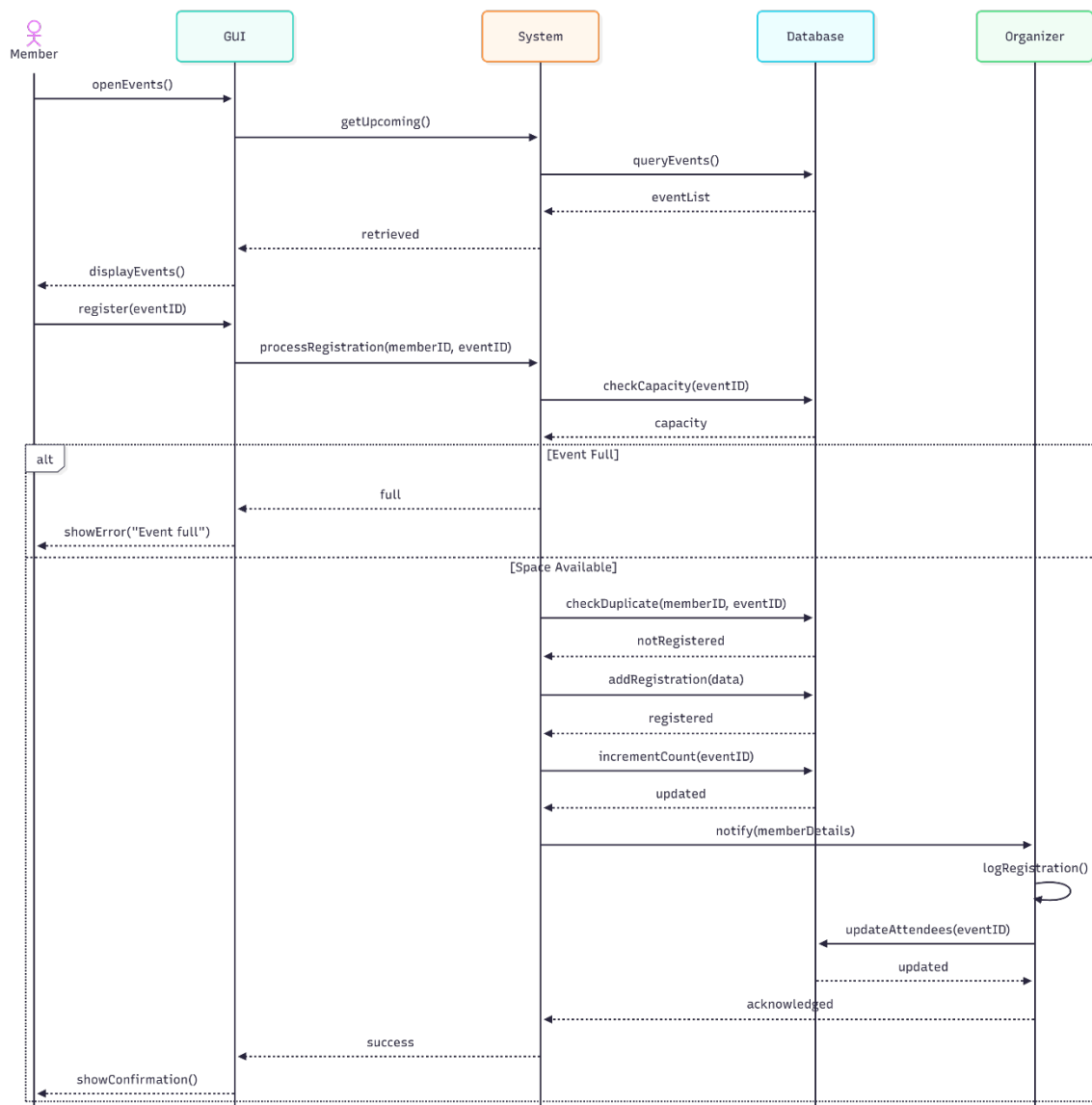
14) AI Product Recommendations

This sequence demonstrates personalized recommendations. When a customer views the homepage, the system retrieves their purchase and browsing history from the database and sends it to the AI Module. The module analyzes patterns and retrieves similar users from the database. For each potential product, it calculates a relevance score by retrieving product metrics from the database, ranks products, and saves scores. The module retrieves details for top products from the database, returns recommendations to the system, which displays them to the customer.



15) Register for Community Event

The event registration sequence shows member signup for community events. The member opens events, and the system retrieves upcoming events from the database. After selecting an event, the system checks capacity in the database. If full, an error is shown. If space exists, the system checks for duplicate registrations, adds a new registration record, and increments the participant count in the database. The organizer is notified, logs the registration, updates attendance in the database, and acknowledges. The member receives confirmation with event details.



Section 4 – User Interface Design

4.0 User Interface Design

A description of the user interface design of the Shared Farm software is presented below. The design focuses on delivering a simple, nature-inspired, and interactive experience for all user roles – farmers, investors, shareholders, delivery partners, and consumers.

4.1 Description of the User Interface

4.1.1 Screen Images

Shared Farm is a cloud-based web and mobile platform that enables users to participate in sustainable farming through digital ownership, investment, and produce ordering. The interface embraces a **light-green and white palette**, reflecting eco-friendliness and farm authenticity. Rounded-edge cards, natural imagery, and clear typography create a calm and trustworthy appearance.

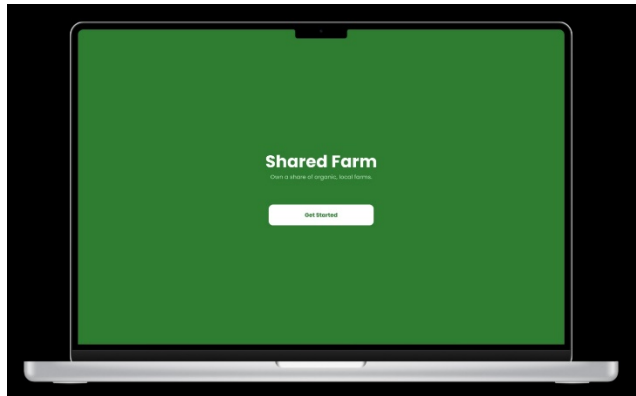
The design emphasizes accessibility and easy navigation between dashboards, product catalogs, and AI insights. All user flows – from registration to subscription, investment, and produce tracking – are visually guided with icons and clear calls to action.

The interface supports multiple roles:

- **Customers** can browse farm products, subscribe to produce boxes, and view environmental impact.
- **Farmers** can manage listings, track stock, and view AI recommendations.
- **Investors/Shareholders** can monitor funding progress, entitlements, and sustainability metrics.

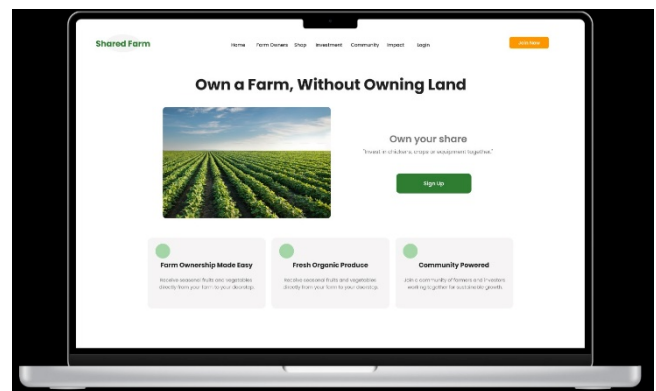
Below are detailed images and descriptions of the GUI. The objects and actions performed when users interact with them are also identified.

4.1.1.1 Splash Screen and Onboarding

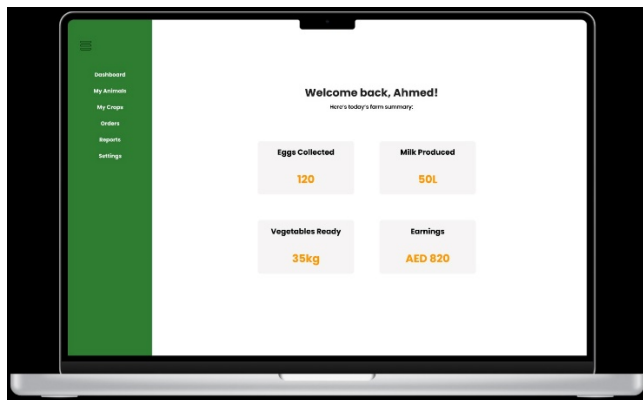


Displays the Shared Farm logo, tagline “*Own a Farm Without Owning Land*”, and a **Get Started** button. It sets the tone for sustainability and community engagement before the user enters the platform.

Introduces the platform’s concept and core benefits such as digital farm ownership, organic produce access, and community support. Offers **Sign Up** and **Learn More** options for first-time users.

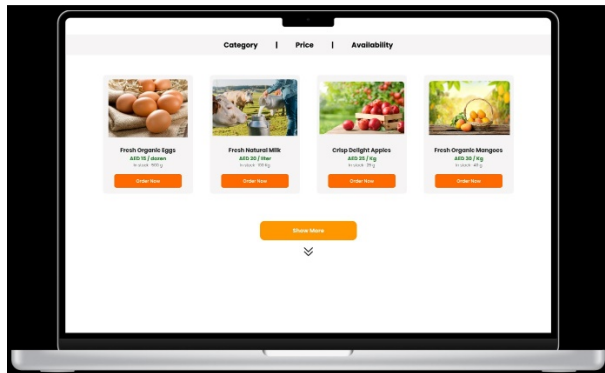


4.1.1.2 Dashboard Pages



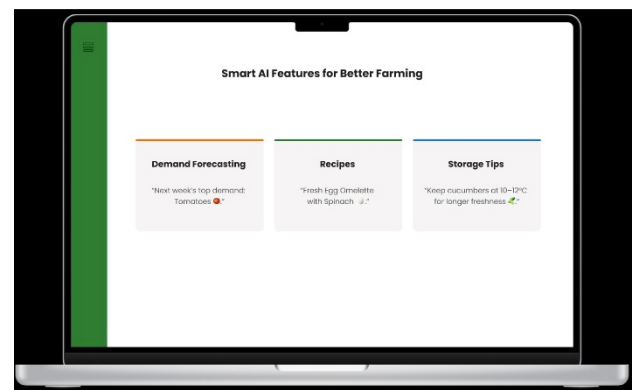
Serves as the personalized control center for each role. Farmers view crop and stock summaries, customers see active orders and rewards, and investors track project funding and ROI metrics in real time.

4.1.1.3 Product & AI Feature Pages

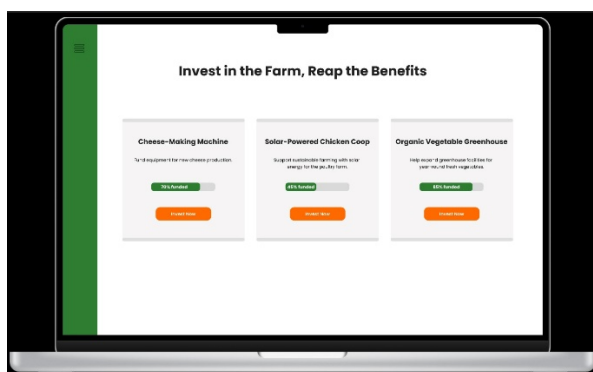


The **Product Catalog Screen** lists farm produce such as eggs, milk, mangoes, and vegetables with images, pricing, and “*Order Now*” buttons. A filter panel allows sorting by category, price, or availability. Hovering over a product reveals its storage tips and origin farm.

The **AI Forecast Panel** visualizes demand predictions, suggesting optimal purchase or restock times. A sub-tab “*Smart Recipes & Storage Advice*” uses the AI Recommendation Engine to promote waste-reduction and better nutrition choices.

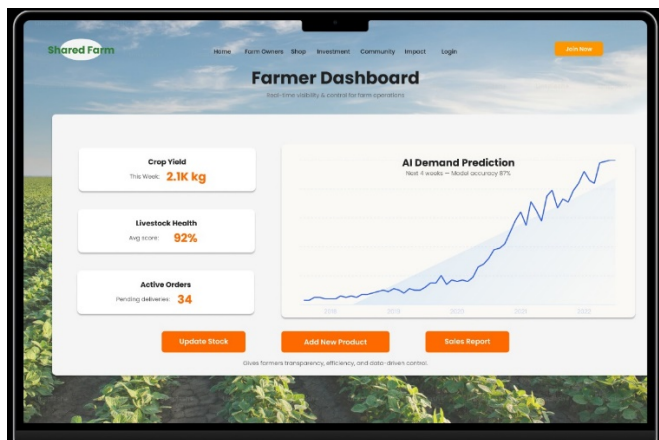
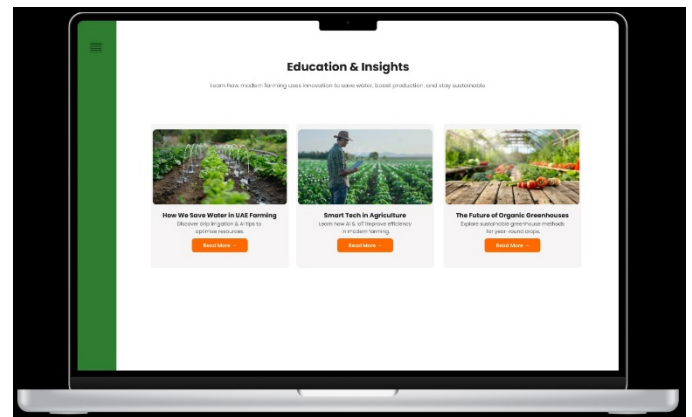


4.1.1.4 Investment and Community Pages



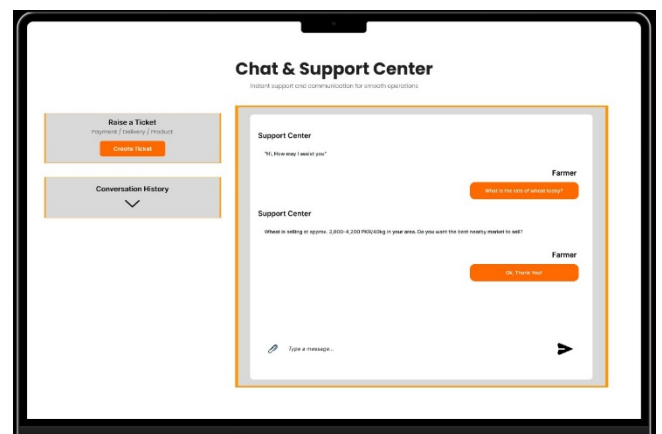
Highlights active farm investment projects with funding progress bars and ROI details. Users can contribute directly through **Fund Now** buttons and track each project’s status and expected returns.

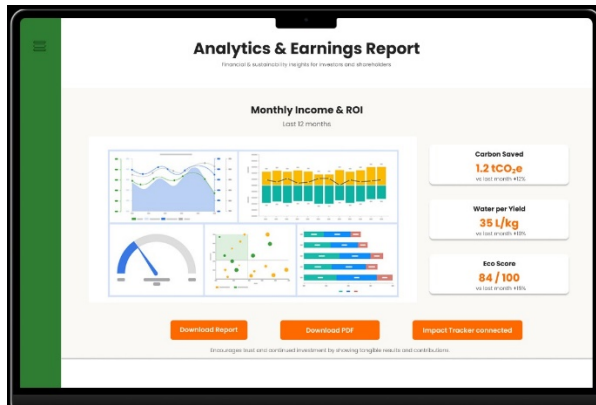
Lists upcoming activities such as *Harvest Festivals*, *Workshops*, and *Farm Tours*. Each event card includes date, location, and **Join Event** or **Register** buttons for participation.



Displays key farm metrics like crop yield, livestock health, and active orders. Includes an AI demand forecast chart and quick actions for updating stock, adding products, and viewing sales reports.

Enables instant communication between farmers and support. Users can raise tickets, view chat history, and get quick help on prices, deliveries, or product issues.





Shows monthly income, ROI, and sustainability metrics such as carbon saved, water use, and eco score. Provides options to download reports or connect the impact tracker.

4.1.2 Objects and Actions

The GUI of Shared Farm comprises multiple elements designed for a unified and effortless user experience:

- **Frames:** Display summary cards for produce, investments, and impact metrics.
- **Role Selector:** Enables login as Customer, Farmer, Investor, or Admin.
- **Pop-Ups and Alerts:** Show success messages, AI notifications, and stock warnings.
- **Menu Bar:** Provides quick access to Catalog, AI Tools, Investments, and Settings.
- **Command Buttons:** Include *Order Now*, *Subscribe*, *Fund Project*, and *View Details*.
- **Scroll Bars:** Allow navigation through product lists and reports.
- **Chat Support Box:** Connects users with customer service or farm managers.
- **Toolbar Actions:** Offer filters, sort options, and data export buttons.
- **Text Fields:** Capture inputs such as search keywords, delivery address, or fund amounts.

All elements work cohesively to ensure an accessible, efficient, and visually coherent experience.

4.2 Interface Design Rules

To ensure the interface meets modern usability standards, Shared Farm adheres to nine core interface design principles:

1. **Continuity:** Consistent use of colors (green and white), fonts, and rounded icons across screens.

2. **Streamline:** Essential features like Catalog and Investments are immediately visible post-login.
3. **Feedback:** Buttons and tabs change color when selected; success and error messages are clear and instant.
4. **Error Control and Prevention:** Input validation ensures correct entries for amounts, emails, and dates.
5. **Visual and Aesthetic Appeal:** Eco-themed layout with minimal clutter and consistent iconography.
6. **Responsiveness:** Optimized for web and mobile screens with adaptive card-based grids.
7. **Clarity of Navigation:** Icons and labels lead directly to Dashboards, Orders, or AI Insights.
8. **Readability and Hierarchy:** Headings use bold green; secondary texts in dark gray for visual separation.
9. **User Control:** Users can customize notifications, language, and theme preferences.

4.3 Components Available

During the initial design phase, Canva was used to draft the UI layouts for screens such as Dashboard, Catalog, and Investment Panels.

In Phase 2, FlutterFlow was adopted to develop an interactive prototype enabling navigation, animations, and basic AI feature integration. This prototype demonstrates core functional flows including ordering produce, funding projects, and tracking AI forecast outputs within a responsive environment.

4.4 UIDS Description

For the User Interface Development System (UIDS), FlutterFlow was employed as the primary framework.

It reduces manual coding effort by providing a visual drag-and-drop environment and integrates with Firebase for real-time data.

This system supports rapid iteration and enhances the UI/UX development cycle by allowing AI widgets (such as forecast cards and recommendation modules) to be inserted directly into the interface without complex back-end code.

The current UIDS prototype is fully navigable, demonstrating key screens such as the AI Forecast Panel, Subscription Page, and Investment Tracker, while maintaining brand consistency and a fluid user experience.

Section 5

5.0 Restrictions, Limitations, and Constraints

This section outlines the major restrictions and constraints that influenced the design and flow of the Shared Farm platform. These limitations shaped architectural choices, user interface decisions, and the implementation strategy.

Major Constraints Affecting Design

- Each component must maintain a **single defined purpose** to avoid overlap or functional interference.
- All modules should remain **lightweight and efficient** in execution time and storage size.
- The **user interface** must be adaptable and intuitive across web, tablet, and mobile devices to serve users of all demographics.
- The system must integrate with **third-party services** such as payment gateways and logistics APIs.
- The platform should gracefully handle disruptions from **external systems** (e.g., broken APIs or delayed gateway responses).
- Data storage and processing components cannot exceed **defined database performance thresholds**.
- **Security** is a fundamental requirement in handling personal, financial, and farm-related data.
- The design must support **scalability** for future integration of AI and IoT sensors.

Major Design Limitations

1. **User Perception of Digital Farming:**

Some users may be skeptical about investing in or subscribing to a virtual farm system due to low awareness of digital agriculture and farm tokenization models.

This created a need for educational content and simplified UX design.

2. **Integration Complexity:**

The application relies on multiple third-party systems (bank APIs, courier services, and AI data feeds). Ensuring compatibility across these services sometimes delayed development.

3. **Scalability and Data Load:**

As the number of users, investments, and transactions grows, the system may encounter response-time degradation. Database optimization and caching mechanisms are required to maintain performance.

4. **Data Privacy and Security:**

Protecting sensitive information such as payment details, personal identification, and investment records is crucial. The platform applies encryption and access-control layers to reduce data breach risks.

5. **AI Dependency and Model Accuracy:**

The reliability of AI predictions for demand forecasting and recommendations depends on historical data quality. Limited or inconsistent data may reduce model precision.

6. **Limited Internet Connectivity in Rural Areas:**

Farmers in remote locations may experience delays syncing data due to weak network access. Offline caching strategies were considered to mitigate this limitation.

Section 6

6.0 Testing Overview

Testing for the Shared Farm system was designed using both **black-box and white-box methodologies**. However, as the development is in the prototype stage, the focus was on black-box testing to validate user-facing functions and ensure correct system behavior without full source-code access.

6.1 Testing Techniques

- **Equivalence Partitioning:** Ensured input values like registration details, payment amounts, and OTP codes fall within valid data classes.
- **Boundary Value Analysis:** Tested edge cases such as maximum subscription limits, product quantities, and investment amounts.
- **Decision Table Testing:** Used for payment approval, entitlement claim validation, and delivery scheduling decisions.

6.2 Expected Software Response

Restriction	User Role	Valid Equivalence Classes	Invalid Equivalence Classes
Password should be 8–15 characters	All users	Farm@2024 / Green#Land12	12345 / FarmSystem2025SuperLong
Email must contain “@” and valid domain	All users	user@sharedfarm.com	user.sharedfarm.com / user@
Investment amount must be ≥ AED 100	Investors	AED 200 / AED 500	AED 50 / AED 0
Product quantity limit per order ≤ 50	Customers	10 units	60 units

Subscription plans must be one per account	Customers	Starter Plan	Starter + Premium
--	-----------	--------------	-------------------

6.3 Types of Testing

1. Unit Testing

- **Expectation:** Each individual module (User, Order, Payment, Entitlement) should perform correctly before integration.
- **Example:** Verify that the *createOrder()* function properly saves data before connecting it with *processPayment()*.

2. Functional Testing

- **Expectation:** Core functions such as AI forecasting, claim processing, and subscription management must align with system requirements.
- **Example:** A user selecting “Family Plan” should receive correct weekly produce allocations.

3. Performance Testing

- **Expectation:** The system should sustain hundreds of simultaneous transactions without timeout.
- **Example:** Stress test with 500 concurrent users making payments.

4. Security Testing

- **Expectation:** User data and payment transactions must remain protected against SQL injection, XSS, or brute-force attempts.
- **Example:** Enforce encryption for all credentials and OTP verifications.

5. Usability Testing

- **Expectation:** Users can navigate dashboards, catalogs, and investments without confusion.
- **Example:** Ensure the AI Feature tab and Impact Tracker are easy to locate and interpret.

6.4 Performance Bounds

- **Response Time:** All page actions (orders, dashboard refresh, AI forecasts) should load within 2–4 seconds.
- **Workload Handling:** The platform supports thousands of concurrent order placements and data updates through load balancing and caching.
- **Latency:** API communications and updates to stock or investment status occur in real time (<3 seconds).
- **Scalability:** Cloud infrastructure ensures elastic server scaling during high-traffic events such as subscription renewals.
- **Fault Tolerance:** Backup servers replicate database entries every hour; recovery time from hardware or network failure is <5 minutes.

6.5 Identification of Critical Components

The following components are considered critical to the successful operation of the Shared Farm platform and must be tested thoroughly to ensure reliability, security, and data integrity:

1. **Payment Component** – Handles all transactions for purchases, investments, and refunds. Any failure could cause financial loss or trust issues.
2. **AI Forecasting System** – Generates demand and stock predictions; errors may lead to over- or under-production.
3. **Entitlement Module** – Manages shareholder allocations and claims; faults can cause incorrect distribution of produce.
4. **Order and Inventory Management** – Tracks orders, deliveries, and stock updates; critical for real-time accuracy.
5. **User Authentication and Access Control** – Secures all logins and permissions; vital for data protection.
6. **Database and Cloud Storage** – Stores user and transaction data; must remain consistent and recoverable after failures.
7. **API Integration Layer** – Connects external gateways and services; interruptions can disrupt core processes.

InnovaCode

These components form the system's backbone and require continuous monitoring, error handling, and redundancy to maintain uninterrupted operation.